HE-OFT: Privacy-Preserving One-Shot Federated Fine-Tuning under Homomorphic Encryption

Technical Report

Halil İbrahim Kanpak, Sinem Sav, Alptekin Küpçü

***Abstract*—Several parties hold data on the same task and want one model built from all of it, without pooling the data. Federated learning does that and concentrates the privacy risk at training time. A one-shot protocol that exchanges a single encrypted contribution removes that surface, but it still ends by handing the trained result to every participant, which is not permitted where the model is itself a regulated or proprietary asset. We present HE-OFT, the first cryptographically secure one-shot federated fine-tuning protocol in which no party receives the trained result. Each client fine-tunes a low-rank adapter and a classifier head on a frozen public backbone, retains the adapter locally, and uploads one encrypted head displacement that the server combines under multiparty CKKS and never decrypts, and a quorum of clients returns only the predicted label, addressed to the party that asked. Across four text classification tasks and one vision task, HE-OFT reaches 0.61 to 0.79 accuracy where a client training alone reaches 0.20 to 0.48, gives up 0.03 to 0.14 against a disclosed model, and answers one query in 400.8 to 1468.3 s on one core, or 48.9 to 176.6 s with GPU acceleration, for 13.5 MiB of traffic.**

***Index Terms*—Federated learning, one-shot federated learning, federated fine-tuning, homomorphic encryption, multiparty computation, privacy-preserving machine learning.**

I. Introduction

Organizations adapt large public pretrained models to their own data by fine-tuning [1], [2]. Often several organizations hold complementary data over the same task, different patient populations, fraud patterns, or customer bases, and each would obtain a better model from the combination than from its own data alone [3]. They cannot pool that data, because data protection law restricts the sharing of confidential records across institutions [4], [5], [6], [7], [8]. Competitive concerns point the same way, and handing the data to whoever operates the fine-tuning infrastructure raises the same objection [9]. Three obstacles stand between that setting and a protocol these organizations could deploy.

Federated learning (FL) enables collaboration [10], and its privacy risk is concentrated at training time [11], [12]. The protocol exchanges model updates instead of data, but the updates are themselves the vulnerability. An adversary aggregating a client's per-round updates mounts membership inference at 25.3% true positive rate at 0.1% false positive rate, against 0.18% from a single snapshot [13]. Two things therefore set the attack surface, namely what a protocol reveals while training runs and how often it reveals it.

Removing that surface takes two measures together. Making the protocol *one-shot*, so that each client contributes once, means that there is no intermediate state for anyone to observe. Encrypting that single contribution means that the one thing that is exchanged is never available in plaintext. In this paper, one-shot does not mean the parties stop communicating. It means that no intermediate training artifact is ever exposed. Prior work applies one of these measures at a time. One-shot FL [14], [15] sends its single contribution in plaintext [16], [17], [18], or protects it with differential privacy [19], [20], [21], which adds noise to the very artifact it releases. Encrypted FL runs many rounds of encrypted training or encrypted aggregation, paying the cryptographic cost on every round, and the direct route of training under encryption forces polynomial replacements of every activation and a deep, repeatedly bootstrapped circuit [22], [23], [24]. That cost scales with the network being trained. POSEIDON trains a three-layer network of sixty-four neurons per layer on handwritten digits at ten parties in 1.4 hours [22], and its successors lower the constant without changing what sets it. Adapting a pretrained transformer by this route is out of reach, and that is the case this paper addresses.

Applying both measures still leaves one artifact exposed, namely the model itself. One-shot protocols end by handing the final model to every participant, which is unacceptable when the model may not be distributed at all. A model fine-tuned on confidential records can be a regulated artifact in high-risk domains such as health, biometrics, and credit [25].

In this work, we present HE-OFT, the first one-shot federated fine-tuning protocol that enables several parties to adapt a pretrained model together and to query the result, without any of them ever holding it in plaintext. The contribution is encrypted under multiparty CKKS [26], [27], a homomorphic encryption scheme whose secret key is split across the parties. The backbone stays public and in plaintext, so the cost of the cryptography does not grow with it. There is no round in which a training artifact is exposed, no plaintext contribution, and no plaintext model at any point in the protocol.

Figure 1 shows how the shared head is built. Every client fine-tunes an adapter and a classifier head on the same frozen public backbone, keeps the adapter, and uploads one encrypted head displacement weighted by the classes it holds. The server adds the ciphertexts and divides by the per-class totals under encryption, so the shared head exists only as a ciphertext and

Corresponding Author: Sinem Sav (sinem.sav@cs.bilkent.edu.tr)
Halil İbrahim Kanpak is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: hkanpak21@ku.edu.tr).
Sinem Sav is with the Department of Computer Engineering, Bilkent University, Ankara, Türkiye (e-mail: sinem.sav@cs.bilkent.edu.tr).
Alptekin Küpçü is with the Department of Computer Engineering, Koç University, İstanbul, Türkiye (e-mail: akupcu@ku.edu.tr).
Manuscript received XX XX, XXXX; revised XX XX, XXXX.

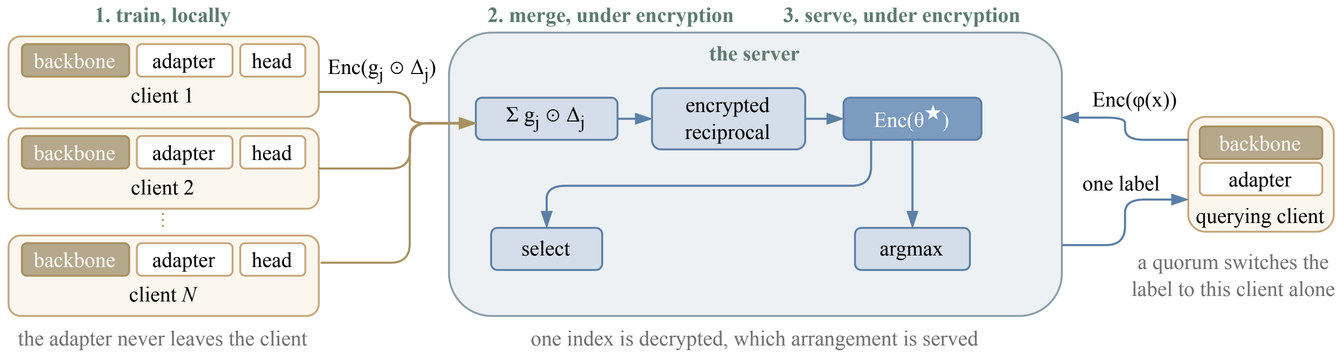

Fig. 1. The protocol in three stages. Each client fine-tunes an adapter and a classifier head on the same frozen public backbone, keeps the adapter, and uploads one encrypted head displacement weighted by its per-class counts. The server adds the ciphertexts and divides by the per-class totals under encryption, holding no decryption key, so the shared head $\theta^\star$ exists only as a ciphertext. The same server answers queries.

the server holds no key to it.

The querier must build its query from parts it can run itself, so those parts must be public. The shared trained quantity therefore sits after the last nonlinearity, which, in a transformer, is a single place. The three contributions below follow from that.

- **A split trainable unit.** Each client keeps its own adapter and never transmits it. Only the classifier head is federated, because a client can improve its own features alone but cannot recognize a class it has never seen. No adapter aggregate exists, so a coalition of all but one client has no sum from which to subtract its own contributions (section III-B).
- **A serving path that never decrypts.** The querying client encrypts its own features, so the server cannot read the query. Only a label leaves the protocol, so a client cannot collect the per-class scores that would let it solve for the head (section III-C).
- **A selection rule that runs under encryption.** Two arrangements are servable at the same cost, the shared head over each client's own adapter and the shared head over the bare public backbone. Each client scoring both on its own held-out data and voting estimates the wrong quantity. The estimator given here corrects that, needs no fitted threshold, and reveals only which arrangement won (section III-D).

We evaluate on four text classification tasks and one vision task, with frozen RoBERTa [28] and vision transformer (ViT) [29] backbones, and we implement the cryptographic protocol in real multiparty CKKS rather than simulating it.

II. Preliminaries

Client indices are $j \in \{1, \dots, N\}$ and class indices are $c \in \{1, \dots, C\}$.

A. Multiparty Homomorphic Encryption

We build on CKKS [26], which performs approximate arithmetic on packed vectors of real numbers under encryption. It is levelled [30], so each multiplication spends one level of a finite budget, and a depleted budget is restored by bootstrapping [31].

TABLE I
Notation.

Symbol	Meaning
N	number of participating clients
$\mathcal{D}_j,\ n_j$	client j's dataset and its size $n_j = \lvert\mathcal{D}_j\rvert$
C	number of classes
$n_{j,c}$	examples of class c held by client j
φ	frozen public backbone, never trained or encrypted
A_j	client j's adapter, trained locally and never transmitted
φ_j	client j's feature map, the backbone composed with A_j
θ_0	public initialiser of the classifier head
h_j	client j's trained head
Δ_j	head displacement $h_j - \theta_0$
w_j	sample weight $n_j / \sum_i n_i$
$\theta^\star$	shared head, held only as a ciphertext
$\mathsf{Enc}(\cdot)$	encryption under the collective public key

A single-key scheme would require one party to hold the secret key and therefore to be trusted with decryption. We use the multiparty variant [27], stated here for a t-out-of-N threshold with $2 \le t \le N$.

That setting asks four things of the scheme, and the protocol uses all four. Key generation produces one collective public key and the evaluation keys from the clients' shares, and never assembles the secret in one place, so any party may compute on ciphertexts and none may read them. Decryption takes partial shares from at least t clients, and any $t-1$ of them learn nothing about a plaintext. A key switch to a designated public key redirects a result so that one chosen party reads it and no other does. A collective refresh restores a spent level budget, and plays the part bootstrapping plays under a single key.

Two consequences follow, and the protocol uses both. Any t clients form a quorum, so serving does not require every client to be online. Any $t-1$ clients are powerless against a ciphertext, so $t-1$ is the corruption bound throughout section IV.

Our implementation instantiates $t = N$, which is the setting the multiparty CKKS library provides, and the measurements of section V-E are taken there. Every statement in section IV holds for general t.

III. Method

The protocol runs in two phases. In the first, each client fine-tunes an adapter and a classifier head on the same frozen public backbone, keeps the adapter, and uploads its head displacement once, weighted by its own per-class counts and encrypted. In the second, a client computes the features of its query itself, encrypts them, and sends them to the server, which applies the encrypted head and reduces the encrypted logits to the index of the largest.

A. Setting

We consider N clients holding private labeled data over a common label space of C classes. Client j holds data $\mathcal{D}_j$ of size n_j. The class proportions differ across clients, and a client may hold no example of a class. The clients want a classifier that works across the whole label space, and they cannot pool their data to build one.

Three requirements separate this setting from ordinary federated learning. (1) The protocol is *one-shot*. Each client uploads once. (2) Confidentiality is *cryptographic*. A client's contribution is never available in plaintext to any other party. (3) The resulting model is *never disclosed*. No party, client or server, ever holds the trained classifier in plaintext.

Each requirement forecloses a family of designs, and the remaining space is narrow. We record the chain of implications here.

C1 *Optimization cannot run under encryption at acceptable cost.* Gradient steps, softmax, and adaptive curvature require deep, repeatedly bootstrapped homomorphic circuits. *Therefore*, for efficiency, all learning happens in plaintext on the clients, and the server performs no learning.

C2 *A server computing on ciphertexts cannot read its inputs.* It cannot inspect, compare, or branch, so it cannot select a data-dependent aggregation rule. *Therefore*, the choice is deferred to the clients, who make it without decrypting anything (section III-D).

C3 *Models trained from different starting points do not combine linearly, and multiplicative depth dominates homomorphic cost.* In the parameter space of a deep network they occupy incomparable regions. *Therefore*, every client starts its trainable unit from one public initializer on one frozen public backbone, and the server combines what the clients return by a linear combination, which applies only after that common start. The clients carry the training and the server's work stays linear in what it receives, which is the cheapest operation the scheme offers and exactly the one the method needs.

C4 *Intermediate training signals are the strongest attack surface.* Each exposed round is a fresh training-time artifact, far more revealing than a finished model (section VI-A). *Therefore*, the protocol is one-shot, and its single artifact is a ciphertext.

C5 *No single party may be able to decrypt.* The parties are mutually distrustful and the server is semi-honest. *Therefore*, the secret key is generated and held distributively, under multiparty CKKS (section II-A).

C6 *The model is never disclosed, yet it must answer queries.* A query is formed from features, and the client that asks the query must be able to compute those features itself. *Therefore*, the client runs the backbone and its own adapter in plaintext, and the trained quantity that is shared cannot sit inside the backbone.
This requirement distinguishes the present design from one in which the model is released, and section III-B derives its consequences.

B. Partitioning the Trainable Unit

a) The construction: Each client uses the same frozen public backbone φ, and trains two things on its own data, a low-rank adapter A_j that adjusts the representation, and a classifier head h_j on top of it. Only the head is shared. Client j encrypts the displacement $\Delta_j = h_j - \theta_0$ of its head from the public initialiser and uploads it once. The server combines the encrypted displacements into a single shared head and never decrypts the result. The adapter is trained locally and never transmitted.

b) Necessity of the partition: The client that sends an inference query must compute the features of that query itself, by **C6**. If the shared trained quantity were an adapter placed inside the backbone, the features would depend on it, and computing them would require evaluating the backbone with encrypted weights. The client would then have to evaluate every nonlinearity of the network homomorphically, which is the cost this design exists to avoid. The trained quantity that is shared must therefore attach after the last nonlinearity, and in a transformer, the only such point is the final linear map. That is the head.

The requirement constrains the *position* of the shared map, not the task it serves. Any trained linear map that sits after the last nonlinearity satisfies it, provided the querying client can compute the features that enter the map. A classifier head is one such map. The vocabulary projection of an autoregressive decoder is another, since it is linear, sits after the final layer normalisation, and takes features the client can compute from a prefix it generated itself. We evaluate classification only. The construction itself does not depend on the label space being a set of classes, and section V-H records what a generation setting would additionally require, including the restriction to greedy decoding that section III-C imposes.

The adapter is relocated. Each client retains its own, so the representation still adapts to local data. The distinction follows from coverage. A client can improve its own representation alone, but it cannot learn to recognise a class it has never observed. Coverage is a property of the head, which is exactly what the federation supplies. Section V-B measures both halves.

Retaining the adapter locally has a second consequence, which is cryptographic. No aggregate of the adapters is ever formed. Section IV states this precisely.

c) The aggregation: Let $g_j \in \mathbb{R}^C$ collect client j's per-class example counts, $g_{j,c} = n_{j,c}$. The shared head is the

Algorithm 1 Building the shared head. All clients hold the same frozen public backbone φ and the same public head initialiser θ_0.

1: clients run distributed key generation, obtaining a collective public key
2: **for** client $j = 1, \ldots, N$ **in parallel do**
3: train adapter A_j and head h_j on $\mathcal{D}_j$ ▷ plaintext, local
4: **keep** A_j, which is never transmitted
5: send $\mathsf{Enc}(g_j \odot \Delta_j)$ and $\mathsf{Enc}(g_j)$, where $\Delta_j = h_j - \theta_0$
6: **end for**
7: **server** adds the ciphertexts to form the numerator and the denominator of eq. (1) ▷ additions only
8: **server** evaluates the reciprocal of the denominator under encryption and multiplies ▷ once, not per query
9: **return** $\mathsf{Enc}(\theta^\star)$, which no party decrypts

coverage-weighted combination

$$\theta^\star \;=\; \theta_0 \;+\; \frac{\sum_{j=1}^{N} g_j \odot \Delta_j}{\sum_{j=1}^{N} g_j}, \tag{1}$$

in which row c of the shared head is the count-weighted average of the clients' row-c displacements. Clients holding a class decide its row, and rows that no client covers remain at the public initialiser. Each client forms the products $g_j \odot \Delta_j$ locally, before encryption, so the server only adds ciphertexts.

d) The division: The denominator of eq. (1) is the vector of federation-wide per-class totals. It cannot be decrypted. Every client knows its own counts, so a coalition of $N-1$ clients that learned the totals would subtract its own and read the remaining client's class histogram exactly. The reciprocal is therefore evaluated under encryption, using an inversion circuit over the positive domain whose refreshes are collective. It is paid once, at aggregation, never per query.

e) Cost: The server performs one ciphertext addition per client per ciphertext, one encrypted reciprocal, and one multiplication. The encrypted object is the head alone, whose size is set by the number of classes and the feature dimension and not by the backbone behind it. Communication is one upload per client and no download of model parameters at all.

C. Inference on the Encrypted Head

a) The construction: A client that wants a prediction computes the features of its query with its own backbone and adapter, encrypts them, and sends the ciphertext to the server. The server applies the encrypted shared head to the encrypted features, obtains encrypted logits, and reduces them to the index of the largest entry under encryption. Algorithm 2 states it.

b) Encryption of the query: The client could send the features in plaintext, which would make the head application cheaper. We encrypt them because features are invertible. A party holding $\varphi_j(x)$ and the public backbone can reconstruct an approximation of x. Sending features in plaintext would therefore hand the server the client's input, which is the thing the protocol exists to protect. The price is that the server applies the head ciphertext by ciphertext rather than ciphertext by plaintext, and section V-E reports what that costs.

Algorithm 2 Answering one query. The shared head remains a ciphertext throughout.

Require: query x at client j; $\mathsf{Enc}(\theta^\star)$ held by the server
1: client j computes $\varphi_j(x)$ with its own backbone and adapter ▷ plaintext, local
2: client j sends $\mathsf{Enc}(\varphi_j(x))$
3: **server** computes $\mathsf{Enc}(\ell) \leftarrow \mathsf{Enc}(\theta^\star)\,\mathsf{Enc}(\varphi_j(x))$ ▷ ciphertext by ciphertext
4: **server** reduces $\mathsf{Enc}(\ell)$ to $\mathsf{Enc}(\hat{y})$, $\hat{y} = \arg\max_c \ell_c$, by a tournament of homomorphic comparisons [36]
5: a quorum of clients key-switches $\mathsf{Enc}(\hat{y})$ to client j's public key
6: **return** $\hat{y}$, readable by client j alone

c) Restriction to the predicted label: Returning logits would defeat the construction. The head is a linear map on features the client computes itself, so a client that reads logits for feature vectors of its choosing recovers the head by solving a linear system in as many queries as the feature dimension [32]. This is not a hypothetical concern. The same object, the final projection layer of a production transformer, has been recovered through a deployed interface for a few tens of dollars, using an interface that exposed log-probabilities and a logit bias, and the operators' response was to restrict that combination [33]. Returning only the index of the largest entry is the same restriction imposed by construction rather than by policy. It places an adversary in the hard-label setting, where functionally equivalent extraction has only recently been achieved for general networks [34], [35]. Those results carry a factor exponential in the number of hidden neurons, and our shared quantity has none, so the restriction buys less here than they suggest. It removes the linear solve and puts a search for decision boundaries in its place, at a factor logarithmic in the precision demanded, and it leaves the head determined only up to a positive scale and a vector added to every row. It does not make extraction impossible, and section IV states the resulting bound as a bound on queries rather than a cryptographic hardness claim.

d) Directed decryption: Decryption needs a quorum, so other clients necessarily participate in producing an answer to a question they did not ask. Rather than decrypt the label collectively, which would reveal it to every participant, the quorum key-switches the ciphertext to the querying client's public key.

e) Cost and the role of the server: The argmax is the one operation in the protocol that is not shallow. Evaluated as a sequential fold of $C-1$ comparisons it costs minutes for a large label space. Packing the logits into one ciphertext and reducing them by a tournament lowers the number of sequential comparisons from $C-1$ to $\lceil \log_2 C \rceil$, and section V-E reports the measured result. None of this is particular to the multiparty setting. Homomorphic evaluation under a collectively generated key is identical to evaluation under a single key, and the schemes differ only in key generation and in decryption, so single-key results carry over. The server is not trusted with anything. It holds the collective public key, the evaluation keys, and ciphertexts, so it computes but cannot decrypt, and it learns neither the model nor any client's data. It is trusted only for availability, and where

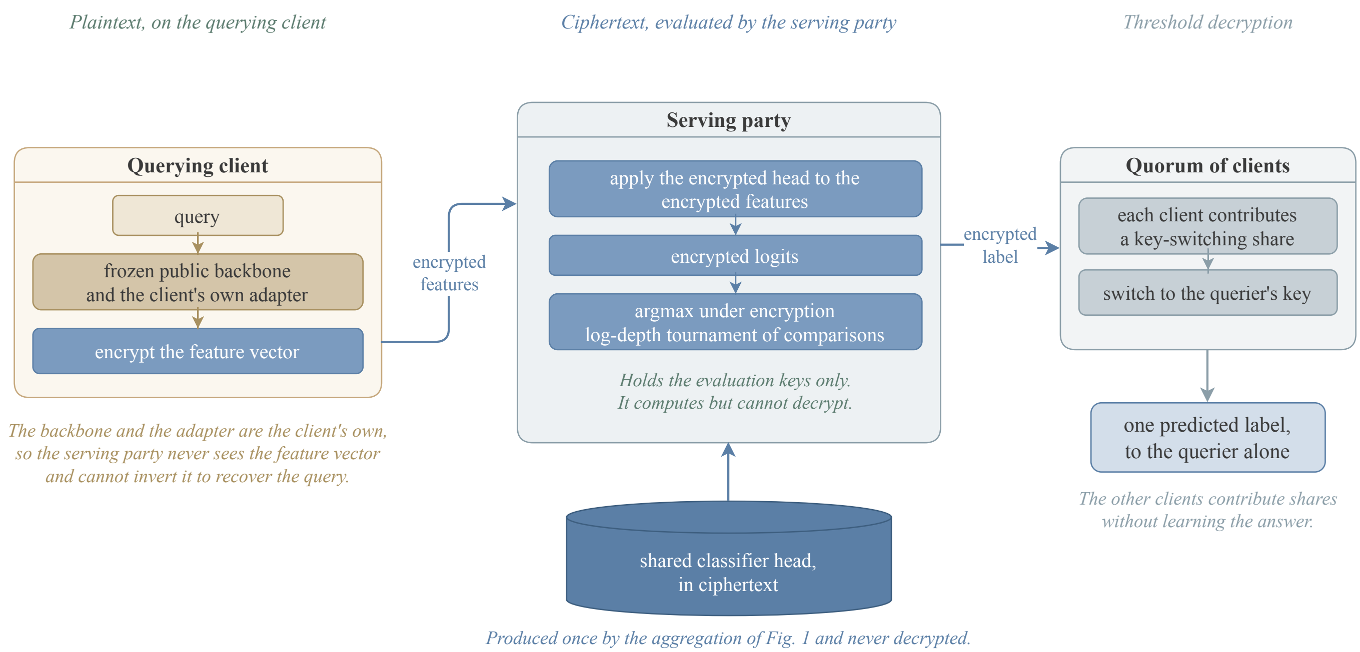

Fig. 2. Answering one query. The querying client computes the features with its own backbone and adapter and encrypts them, so the server never sees the feature vector and cannot invert it to recover the query. The server holds the evaluation keys only. It applies the encrypted head, obtains encrypted logits, and reduces them to the index of the largest entry by a tournament of homomorphic comparisons. A quorum of clients then switches that index to the querying client's key, so the other clients contribute shares without learning the answer.

honest computation must be certified, verifiable homomorphic encryption applies [37].

f) Scope: This construction serves a trained quantity that is linear in the features. Serving a trained quantity that sits inside the backbone is the problem that secure transformer inference addresses, and an encrypted shared adapter composes with any such system at that literature's cost. We do not re-solve it. One difference should be noted. Those systems evaluate a plaintext model on encrypted inputs, whereas here the shared weights are encrypted as well, so each adapter site would carry a ciphertext-by-ciphertext product. The dominant cost in that literature is the nonlinearities, which are unchanged, so the added cost is the low-rank product per site rather than a different order of magnitude.

D. Candidate Selection Under Encryption

a) The construction: Two arrangements are servable at the same cost, the shared head over each client's own adapter, and the shared head over the bare public backbone with no adapter at all. Which is better depends on the task, and the federation must choose without decrypting either. Each client evaluates both on data it holds back from its own training, entirely under encryption, and reports encrypted per-class counts. The counts are combined into one encrypted score per arrangement, and exactly one value is decrypted, which arrangement the federation adopts.

b) Inadequacy of the held-out vote: The natural procedure is for each client to measure the accuracy of both arrangements on its held-out data and vote. The procedure estimates the wrong quantity. The arrangement without an adapter is a single model, and the union of the clients' data is the whole training set, so pooled held-out measurements estimate its accuracy on the global distribution closely. The arrangement with personal adapters is N different models, and client j's held-out data can only measure client j's own model. How client j's model performs on the other clients' distributions is not observable locally, and that is precisely where it fails. A client's held-out set also contains only the classes that client holds, so it cannot measure performance on classes the client has never seen. The measurement is systematically optimistic for the personal arrangement, and section V-D shows that the resulting vote chooses wrongly whenever the two differ.

c) A prior-weighted estimator: Score both arrangements as the expected accuracy of a prediction on an example drawn from the federation's own class distribution,

$$E \;=\; \sum_{c=1}^{C} p_c\, a_c, \qquad p_c \;=\; \frac{\sum_j n_{j,c}}{\sum_j n_j}, \tag{2}$$

where a_c is the accuracy on class c and p_c is that class's share of the federation's examples, taken from the per-class counts already used in eq. (1). For the shared arrangement there is one model, so per-class evidence from different clients concerns the same object and combines. For the personal arrangement each client's evidence concerns only its own model, and classes it does not hold contribute no evidence at all. Scoring those classes at zero makes E a lower bound for the personal arrangement. There is no fitted threshold anywhere.

d) Requirements on the held-out split: The estimator needs an estimate of accuracy on classes a client has not seen, and its only local proxy is accuracy on classes it has barely seen. A held-out set carved uniformly almost never contains

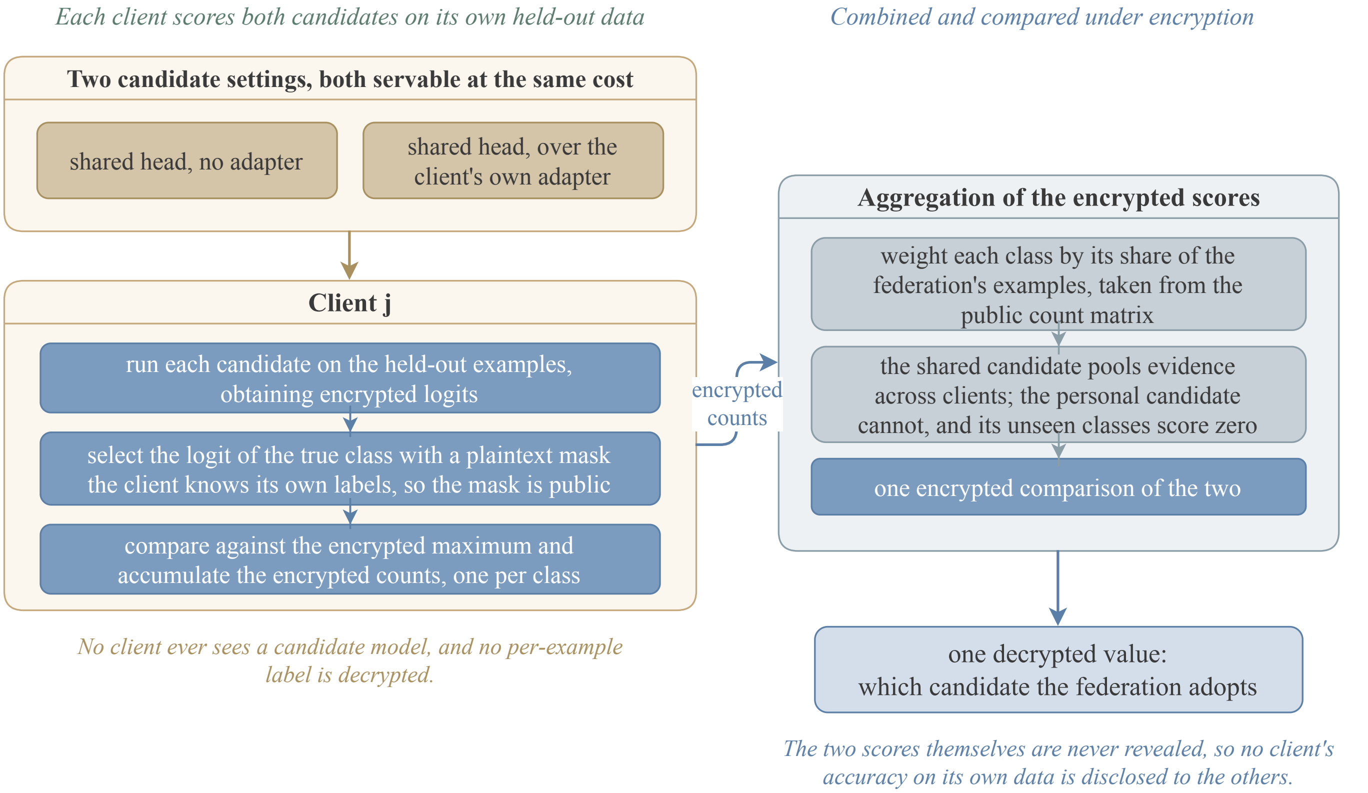

Fig. 3. Choosing between two arrangements without decrypting either. Each client scores both on data held back from its own training, selecting the logit of the true class with a plaintext mask, which is public because the client knows its own labels. The per-class counts are weighted by each class's share of the federation's examples, taken from the count matrix already used for the aggregation. The shared arrangement pools evidence across clients because it is one model; the personal arrangement cannot, and classes a client does not hold contribute nothing, which makes its score a lower bound. Exactly one value is decrypted.

Algorithm 3 Choosing an arrangement without decrypting either. Nothing is revealed except the index of the winner.

for each arrangement m and each client j **do**
  for each held-out example (x, y) of client j **do**
    obtain $\mathsf{Enc}(\ell)$ and $\mathsf{Enc}(\max_c \ell_c)$ as in algorithm 2
    select $\mathsf{Enc}(\ell_y)$ with a plaintext mask ▷ the client knows y, so the mask is public
    add $\mathsf{Enc}\big(\mathbf{1}[\ell_y = \max_c \ell_c]\big)$ to the class-y accumulator
  end for
end for
combine the accumulators into $\mathsf{Enc}(E_m)$ by eq. (2), using public scalars ▷ depth one
compare the two encrypted scores
return the decrypted index of the larger ▷ one value

a class the client holds one or two examples of, so that proxy is unmeasurable. Each client therefore reserves at least one held-out example from every class it holds before carving the remainder at random. A client holding a single example of a class donates it, so its adapter never trains on that class, which is exactly the probe the estimator needs. This costs a negligible amount of training data and requires no change to the cryptography.

e) Cost and disclosure: Every step is depth one except the comparisons, which are the same circuit already used for the argmax. One value is decrypted for the whole federation. The per-client per-class accuracies are not decrypted, and must not be.

The threat model, the view each party has and what a coalition of clients can learn, is stated in section IV.

IV. Security

The participants are N clients and a server. The server is semi-honest. It follows the protocol and may try to infer private information from what it observes. Clients may likewise be semi-honest and may collude. Section IV-C strengthens the client side of this model, and allows up to $t-1$ clients to deviate arbitrarily while the server stays honest. Section IV-D explains why that assumption cannot be dropped.

a) View of the server: Ciphertexts under the collective public key, the public per-client sample counts, and the evaluation keys. It holds no decryption key. The server additionally sees encrypted query features, which it cannot read.

The design is arranged so that no arithmetic shortcut evades it. No aggregate of the adapters exists, so there is no sum from which a coalition can subtract its own contributions. The shared head is never decrypted, so it cannot be inverted the same way. The per-class totals are never decrypted, so a coalition cannot recover the remaining client's class histogram.

That last sentence is worth stating precisely, because it does not say the histogram is unrecoverable. It says the protocol publishes no direct read of it. A client's counts also shape the displacement it uploads, since for a linear map trained under

Functionality 1 HE-OFT $\mathcal{F}$

Parameters. The client count N, the decryption threshold t, the per-client query allowance Q, and the label space size C. The parties are clients $P_1, \dots, P_N$ and a server S, which is not part of $\mathcal{F}$. The functionality holds a counter q_j for each client, each initialised to 0, and a store that is initially empty and that $\mathcal{F}$ never sends to any party.

Inputs. Each client P_j inputs its dataset $\mathcal{D}_j$, and later inputs queries x. The server S inputs a selection request.

Outputs. Every party receives the selected index $a^\star$ and the phase signals. Client P_j receives one label for each of its first Q queries, and $\perp$ afterwards.

Steps.

1) *Training and aggregation.* On $(\mathsf{Train}, \mathcal{D}_j)$ from every client, run the local training of section III-B on each $\mathcal{D}_j$ in both servable arrangements and store the two shared heads $\theta^\star_A$ and $\theta^\star_B$ of eq. (1). Send $(\mathsf{Done}, \mathsf{train})$ to every party.
2) *Selection.* On (Select) from S, evaluate the estimator of eq. (2) on each of the two stored heads, send the index $a^\star \in \{A, B\}$ of the larger to every party, and send $(\mathsf{Done}, \mathsf{select})$ to every party.
3) *Serving.* On (Query, x) from client P_j, return $\perp$ to P_j if $q_j \geq Q$. Otherwise set $q_j \leftarrow q_j + 1$, return $\hat{y} = \arg\max_c (\theta^\star_{a^\star}\, \varphi_j^{a^\star}(x))_c$ to P_j alone, where $\varphi_j^A = \varphi$ is the public backbone and $\varphi_j^B = \varphi_j$ is the backbone composed with P_j's adapter, and send $(\mathsf{Done}, \mathsf{query}, j)$ to S.

Leakage. $\mathcal{F}$ reveals $\mathcal{L}$ to the adversary, consisting of the public parameters, the per-client sample counts n_j, the announced index $a^\star$, and the fact that each Done message was sent.

cross-entropy the magnitude of the row for a class grows with the number of examples of that class the client held [38]. Those displacements are ciphertexts and the merged head is a ciphertext, so nothing is readable while the encryption stands, and section IV-F1 measures what survives into an extracted copy. What inverting the totals under encryption buys is the removal of the one channel that would have handed the histogram over in the clear.

This section states what the protocol guarantees. Section IV-A gives an ideal functionality. Section IV-B proves that the protocol realizes it against semi-honest parties. Section IV-C bounds what a malicious client coalition learns about an honest client's data. Section IV-D shows that the server must be honest and what would remove that requirement. Section IV-E separates what the cryptography guarantees from what the task itself leaks.

A. The Ideal Functionality

Functionality 1 defines the ideal functionality $\mathcal{F}$ against which we measure the protocol. The server is an entity outside $\mathcal{F}$, so it may be corrupt. The form of the functionality follows the secure aggregation functionality of ELSA [39], and placing the aggregator outside it follows FULLSA [40].

The sample counts sit in the leakage because the protocol uses them as public scalars. We prove security relative to both.

Three properties of $\mathcal{F}$ are what the protocol must reproduce. No party ever holds either shared head. The only value the whole federation learns is one index. A client learns the labels it asks for, up to its allowance, and nothing else.

B. Security Against Semi-Honest Parties

Definition 1 (Semi-honest security)**.** Let $\mathcal{A}$ corrupt a set of parties that may contain the server and at most $t-1$ clients. Write $\mathsf{view}^{\Pi}_{\mathcal{A}}(\{\mathcal{D}_j\}_{j=1}^N)$ for the corrupted parties' joint view of a protocol run on datasets $\{\mathcal{D}_j\}_{j=1}^N$. The protocol Π securely realizes $\mathcal{F}$ with leakage $\mathcal{L}$ if there is a probabilistic polynomial-time simulator $\mathcal{S}$ such that, for every $\{\mathcal{D}_j\}_{j=1}^N$,

$$\mathsf{view}^{\Pi}_{\mathcal{A}}(\{\mathcal{D}_j\}_{j=1}^N) \overset{c}{\approx} \mathcal{S}\big(\mathcal{L},\ \{\mathcal{D}_j\}_{j\ \mathrm{corrupt}},\ \mathsf{out}_{\mathcal{A}}\big),$$

where $\mathsf{out}_{\mathcal{A}}$ collects the answers $\mathcal{F}$ returns to the corrupted clients.

The simulator receives the answers because the protocol delivers them. A definition that withheld them would be unachievable, and the theorem would say nothing about a system that answers questions.

Theorem 1 (Semi-honest security)**.** *Assume the t-out-of-N threshold CKKS scheme is secure under chosen plaintext attack (IND-CPA) against an adversary holding $t-1$ key shares. Then the protocol securely realizes $\mathcal{F}$ with leakage $\mathcal{L}$ against any semi-honest adversary corrupting the server and at most $t-1$ clients, in the sense of definition 1.*

Proof sketch. The simulator is given $\mathcal{L}$, the corrupted clients' datasets, and the answers the corrupted clients receive.

Setup. The simulator runs the distributed key generation with the corrupted parties, playing the honest clients on uniformly chosen shares.

Training. For each honest client the simulator sends an encryption of zero in place of $\mathsf{Enc}(g_j \odot \Delta_j)$, once for each of the two arrangements, and in place of $\mathsf{Enc}(g_j)$, which the two share. The number of ciphertexts follows from the public parameters, so the simulator knows it. A hybrid argument replaces the honest ciphertexts one at a time, and each neighbouring pair of hybrids is indistinguishable by IND-CPA against $t-1$ shares, the assumption of the theorem.

The server's additions and its multiplication by the reciprocal are deterministic functions of those ciphertexts and of public scalars, so the simulator computes them as the server does. The encrypted reciprocal is not such a function. Its inversion circuit consumes levels and restores them by collective refresh, an interactive protocol in which the participating clients contribute fresh randomness. The simulator therefore invokes the simulator of the refresh protocol for each of the two refreshes, playing the honest clients on that protocol's simulated shares. Both refreshes sit at fixed points in the circuit that do not depend on any plaintext, so the simulator knows their number and position in advance.

Selection and serving. The selection step decrypts one value. The simulator takes that index from $\mathcal{L}$ and produces the collective key-switching transcript for it from that protocol's simulator. For each query of a corrupted client, the simulator holds the answer in $\mathsf{out}_{\mathcal{A}}$ and simulates the key switch to that client in the same way. Queries by honest clients are switched to keys the adversary does not hold, so their transcripts are simulated as key switches of encryptions of zero.

Composing the three parts gives a view computationally indistinguishable from the real one. □

C. Input Privacy Against Malicious Clients

Theorem 1 assumes every party follows the protocol. We now let the clients deviate. We bound what a deviating coalition learns about the data of a client that does not deviate. We do not claim correctness.

Definition 2 (Content game)**.** The adversary $\mathcal{A}$ corrupts $t-1$ clients. The server is honest. At least one client h stays honest.

1) $\mathcal{A}$ outputs two datasets D_0 and D_1 with $|D_0| = |D_1|$.
2) The challenger samples $b \in \{0,1\}$ and gives D_b to client h.
3) The protocol runs. $\mathcal{A}$ may deviate arbitrarily in the values its clients upload, in the shares they contribute, and in the queries they ask.
4) $\mathcal{A}$ outputs b'. Its advantage is $|\Pr[b' = b] - \frac{1}{2}|$.

The equal-size restriction is necessary. The per-client sample counts are public, so an adversary free to choose datasets of different sizes reads the answer off the counts without attacking anything.

Theorem 2 (Input privacy)**.** *Assume the conditions of theorem 1. Then the advantage of $\mathcal{A}$ in definition 2 is at most $\mathrm{negl}(\lambda) + \delta(Q_{\mathrm{tot}})$, where $Q_{\mathrm{tot}} = (t-1)Q$ and δ is the advantage available from Q_{tot} label queries to the served head.*

Proof sketch. The adversary's view consists of the key-generation transcript, the honest clients' uploaded ciphertexts, the values the server computes, and the answers to its own queries.

The first four are independent of b up to $\mathrm{negl}(\lambda)$. The honest ciphertexts are indistinguishable from encryptions of zero by the hybrid argument of theorem 1, which holds against an adversary with $t-1$ shares. The server is honest, so the values it computes are the prescribed deterministic functions of those ciphertexts and of public scalars. The server is honest, so every ciphertext presented for key switching is the prescribed output of algorithm 2 on a query, and its plaintext is a label the coalition does not choose. Deviating in the uploaded values does not help either, because the adversary already chooses those values in the real protocol and they carry no dependence on b. Deviating in the contributed shares does not help, because a share that is not the prescribed one makes the reconstruction fail, which aborts the query and returns nothing.

What remains is the answers. These do depend on b, because $\theta^\star$ depends on client h's displacement, and it is bounded by what Q_{tot} label queries reveal. □

The bound scales with the corruption threshold. A coalition of $t-1$ clients commands $(t-1)Q$ queries whatever the size of the federation. Section V-F measures δ and turns it into a bound on Q.

D. Necessity of an Honest Server

Theorem 2 keeps the server honest. That assumption is necessary. Decryption is a protocol the clients run on a ciphertext the server presents. A deviating server can present a ciphertext of its own choosing, such as a row of $\theta^\star$, rather than the output of algorithm 2. The natural repair is to have an honest client refuse such a request. It cannot, and the obstacle is the encryption itself.

Proposition 1 (No plaintext gate from a ciphertext)**.** *Let an honest client decide, by a polynomial-time predicate D on its view, whether to contribute its decryption share for a presented ciphertext. Let the rest of that view be independent of the underlying plaintext. If the scheme is IND-CPA against an adversary holding $t-1$ key shares, then for every pair of plaintexts m_0 and m_1,*

$$\left|\Pr[D(\mathsf{Enc}(m_0)) = 1] - \Pr[D(\mathsf{Enc}(m_1)) = 1]\right| \le \mathrm{negl}(\lambda).$$

Proof. A gap between the two probabilities gives an IND-CPA distinguisher that holds one key share. One share is at most $t-1$ shares, so the assumed security of the scheme applies directly. □

An honest client therefore releases its share for both plaintexts or for neither. No protocol of this message pattern realizes $\mathcal{F}$ against a malicious server, which is why theorem 2 places the server among the honest parties.

Two mechanisms would remove the assumption, and we implement neither. The first checks provenance rather than content. The serving circuit is a deterministic function of the encrypted head, the encrypted query, the evaluation keys and the public parameters, and its level restoration is deterministic as well, because the protocol restores levels under collectively generated bootstrapping keys rather than by collective refresh. A client can therefore recompute the prescribed output ciphertext and compare it as a string. The design choice that saves 510 MiB of traffic on every query is the same one that leaves the circuit verifiable. Checking a random fraction p of queries suffices, because recovering the shared map needs on the order of Cd deviating requests and a campaign of that length escapes a single checker with probability at most $(1-p)^{Cd}$, which at $p = 0.01$ is below 10^{-13} on the smallest of our tasks. The second mechanism certifies which function the server evaluated, through a zero-knowledge proof of correct evaluation or verifiable homomorphic encryption [37]. Section V-H records both as limitations.

E. Leakage Inherent to the Functionality

Theorem 1 shows the protocol reveals no more than $\mathcal{F}$ reveals. That leaves the question of what $\mathcal{F}$ itself reveals, and the answer is not nothing.

$\mathcal{F}$ answers queries. Answers carry information about $\theta^\star$, and enough of them reconstruct it. Section V-F measures the rate at three to five queries per parameter. This is a property of the functionality. Any protocol that realizes $\mathcal{F}$ inherits it.

The cryptographic claim is that the protocol adds nothing to the leakage of $\mathcal{F}$. The operational claim is that $\mathcal{F}$'s own leakage is bounded by the query allowance Q. The first rests on IND-CPA. The second rests on a measurement.

F. What a Fellow Client Learns

Theorem 2 bounds a deviating coalition by $\mathrm{negl}(\lambda) + \delta(Q_{\mathrm{tot}})$ and leaves δ unnamed. We name it here. A coalition of clients is the one adversary the cryptography does not silence, because its members hold key shares, run the backbone themselves, and receive answers by design.

A client outside the coalition contributes four things to what the coalition sees. It contributes the public sample count n_j, which the protocol uses as a public scalar. It contributes the ciphertexts it uploads, which are computationally independent of what they encrypt. It contributes its share of the announced index $a^\star$ and of the phase signals. It contributes its part of every answer the coalition is given. It contributes no per-class total, because section III-B inverts those under encryption and never decrypts them, and it contributes no adapter, because no adapter aggregate exists.

The allowance alone does not bound what the answers reveal. Membership inference from predicted labels is an established attack on a classifier that answers queries [41], and its cost is not a function the allowance controls. We therefore bound the channel at the head rather than at the budget.

Proposition 2 (The query channel is no stronger than the head)**.** *Let $\mathcal{A}$ corrupt at most $t-1$ clients in definition 2, with the server honest, and let it ask at most $Q_{\mathrm{tot}} = (t-1)Q$ queries. Then there is an adversary $\mathcal{B}$ that receives both shared heads in plaintext together with $\mathcal{L}$, asks no query, and satisfies*

$$\mathsf{Adv}(\mathcal{A}) \ \leq \ \mathsf{Adv}(\mathcal{B}) + \mathrm{negl}(\lambda).$$

Proof. The proof of theorem 2 splits the view of $\mathcal{A}$ into the key-generation transcript, the honest clients' ciphertexts, the values the honest server computes, and the answers to its own queries. The first three are independent of b up to $\mathrm{negl}(\lambda)$, and $\mathcal{B}$ produces them from $\mathcal{L}$ and from encryptions of zero exactly as the simulator of theorem 1 does.

Each answer is $\arg\max_c(\theta^\star_{a^\star}\, \varphi_j^{a^\star}(x_i))_c$ for a query x_i that $\mathcal{A}$ asks at one of its own clients j. Under $a^\star = A$ the map is the public backbone. Under $a^\star = B$ it composes the backbone with the adapter of client j, and that client is corrupt, so $\mathcal{B}$ holds the adapter. $\mathcal{B}$ holds both heads by assumption and reads $a^\star$ from $\mathcal{L}$. It computes every answer itself.

$\mathcal{B}$ therefore runs $\mathcal{A}$, answers each query as the server would, and returns whatever $\mathcal{A}$ returns. The simulation is exact outside the negligible event on which the first three parts of the view differ. □

Write δ_{wb} for the advantage of the best polynomial-time adversary that holds both shared heads. Proposition 2 gives $\delta(Q_{\mathrm{tot}}) \leq \delta_{\mathrm{wb}}$ at every budget. What a coalition learns about an honest client is capped by what the shared head itself reveals, and that cap does not depend on the allowance. The allowance prices the cost of approaching the cap, and section V-F measures that cost.

Two consequences follow for a deployment. Lowering the threshold t raises Q_{tot} and moves a coalition along the curve toward the cap, and it does not move the cap. Raising the allowance does the same. Neither choice changes what is ultimately learnable, so both are decisions about cost.

1) Measuring the cap: We measure δ_{wb} and find it at chance. The adversary in proposition 2 receives both shared heads in plaintext, so we give an attacker exactly that and run the likelihood-ratio membership attack of Carlini et al. [42] against it. We build 64 shadow federations per cell, each one N clients training on half of their own examples followed by the merge of eq. (1), and we test 2000 candidates over three tasks, three seeds and both arrangements. The attack reaches an area under the curve between 0.49 and 0.53, and a true-positive rate between 0.008 and 0.023 at a false-positive rate of 0.01. A true-positive rate equal to the false-positive rate is chance.

Handing the head over is a counterfactual, and that is the point of measuring there. No party ever holds either head, so this is the world in which the cryptography has already failed. Proposition 2 says the query channel is no stronger than that world. Measuring the strongest adversary the proposition admits therefore bounds every adversary the protocol admits, and the bound is chance.

A coalition that extracts a copy first does no better. We fit a copy of the head to 2×10^5 label-only answers, the largest budget of section V-F, and attack the copy. The area under the curve moves to between 0.49 and 0.50, and the true-positive rate at a false-positive rate of 0.01 to between 0.009 and 0.017. Extraction costs the attacker queries and returns nothing, which is what proposition 2 predicts and what no published measurement had shown.

The strongest label-only attack we found does no better, and it fails for a reason worth naming. OSLO crafts one query per candidate rather than reading a statistic off a model, and it reports a true-positive rate of 6.7 per cent at a false-positive rate of 0.001 on CIFAR-100 where the previous best label-only attack reports 0.3 [43]. Our serving interface is more exposed to it than an ordinary one, because a client uploads its own encrypted feature vector and theorem 2 lets a coalition deviate in the queries it asks, so the crafted point enters directly. Over eighteen cells the attack reaches an area under the curve between 0.47 and 0.52.

The reason sits in the attack's own calibration. A coalition chooses its one perturbation on data it holds, where it knows for every surrogate which examples that surrogate trained on. The best separation it achieves there is 0.065 at most and below 0.01 in most cells. The crafted query does not fail to transfer to the served head. The difference in margin between a member and a non-member is not there to transfer.

We report one figure we do not treat as a result. Predicting membership from whether the head classifies an example correctly reaches an area under the curve of 0.72 on Banking77. Our holdout reserves at least one example of every class a client holds, so held-out examples are richer in rare classes than trained ones, and that rule separates them for a reason that has nothing to do with membership. The likelihood-ratio attack calibrates each example against its own shadow distributions and does not inherit the confound.

The coverage-weighted merge invites a sharper attack and does not yield to it. A class that one client alone holds takes its row from that client's displacement, with no averaging over the federation, and for a linear map with a bias the ratio of the row displacement to the bias displacement is a weighted mean of that client's features [44]. We compute that ratio for all 570 class rows of the three tasks. It matches the closest example of its own class better than the closest example of another class by 0.012 in cosine where one client holds the class, and by 0.006 where six or more do, against an absolute cosine of 0.16 and a random-direction baseline of 0.029. The row points into the space the features occupy. It does not point at a class, and it does not point at a record.

Three other attack families ask for something this construction does not provide, and we say which and why. Reconstruction from weights is the first. The strongest published attack in our exact setting recovers training images from heads trained on frozen 768-dimensional embeddings, and its authors report that it fails on linear probes and name linear models as a mitigation [45]. Their own governing ratio is the parameter count over the unknowns, which for a linear head reduces to the class count over the client's sample count. Ours is about 0.01, and the smallest ratio at which they recover anything is 0.5.

Attribute inference is the second. Against a model trained on features of this kind, the black-box attack performs at the base rate, 0.28 against a base rate of 0.28 on one dataset and 0.05 against 0.06 on another [46]. The variant that beats imputation reads individual hidden-layer activations, and a single linear map has none.

Gradient inversion is the third, and it does not instantiate at all. Every attack in that line needs gradients with respect to parameters that see the input, or a model the server chose [47], [48]. The backbone here is public and fixed, every client runs the same one, and the only uploaded object is a trained displacement of a map that sits after the last nonlinearity.

Two limits belong with these numbers. A thousand members resolve a true-positive rate no finer than 0.001, so every figure we report at a false-positive rate of 0.001 counts two to four examples rather than a rate. And the shadow federations vary who trains on what while holding the coverage pattern fixed, so they say nothing about how leakage moves when the partition moves.

Read together, the three surfaces say one thing. Even where extraction has already succeeded, and even where we hand the head over outright, the membership attack of Carlini et al. [42] stays near chance. The head is a linear map fitted for two hundred steps over thousands of examples per class, and the coverage-weighted merge gives a coalition no row it can read.

G. What a Decryption Carries

The protocol decrypts twice. The selection step decrypts one scalar, and each answer is key-switched to the client that asked for it. Both are intended outputs. In an exact scheme that would end the discussion, because a decryption returns the message and nothing else. CKKS is approximate, so a decryption returns the message together with an error term, and that term is a function of the secret key and of the values the circuit consumed. Li and Micciancio show that an adversary which observes decryption results recovers the key that produced them [49].

Their attack does not need the message. It reads the ring element $c_0 + c_1 s$ out of the decoded output and inverts it, so an argument that the adversary cannot predict what the circuit should have produced does not answer it. The reason the attack does not reach us is structural. It recovers whichever key performed the decryption. Every decryption here happens after a collective key switch that moves the ciphertext off the collective key, so the key it recovers is the querying client's own, which that client already holds. The collective key s never appears in a relation the coalition can invert, and $\mathsf{Enc}(\theta^\star)$ is never decrypted at all.

The published threshold attack does not reach us either, for a separate reason. It identifies ciphertexts whose noise is exactly zero and reads a linear equation in the key off each one [50]. Detecting a zero-noise ciphertext requires a rounding step whose success can be observed, which exact schemes have and CKKS does not. Their statement about threshold CKKS covers the case where no smudging is applied.

What remains is a condition rather than an attack, and we state it because our parameters do not meet it. The key-switch simulator of the multiparty scheme we build on is correct when the smudging noise dominates the noise of the ciphertext being switched, which is what lets it produce an honest client's share without knowing that noise [27]. That scheme sets the requirement at $2^{\lambda/2}$ times the ciphertext noise. Our implementation smudges at eight times the fresh-encryption noise, which is the value the library uses in its own tests, and the serving circuit leaves a ciphertext noise several orders of magnitude larger than that. The simulation step of theorem 1 therefore rests on a hypothesis our measured configuration does not satisfy. What the step would otherwise hide is the circuit noise, which depends on the honest clients' displacements. Whether that is exploitable is open, and the scheme's authors record it as open as well.

Meeting the stated requirement is not a matter of raising one constant. At a plaintext scale of 2^{45} the required smudging exceeds the scale itself, so it would destroy the answer rather than protect it. A configuration that satisfies the condition needs a larger scale and modulus, and every cost in section V-E would move with them. The tension is recorded in the literature rather than particular to us. Threshold CKKS is insecure under this notion when no smudging is applied, and flooding sized to the rule is expected to cost the precision of the decrypted result [50]. We report the configuration we measured and state the gap rather than claim a guarantee we did not instantiate.

Repeating a query turns that gap into a key recovery, and we state the arithmetic because it decides which control the deployment needs. Write the served ciphertext as (c_0, c_1) under the collective key s. The querier decrypts $c_0 + c_1 s + E$, where E collects the honest parties' smudging terms. The serving circuit is deterministic, so a client that resubmits an identical query receives the same c_0 and c_1 and a freshly drawn E. Averaging k repetitions drives E to zero and converges on $c_0 + c_1 s$ exactly. The noise of the ciphertext is not a term outside that expression. It is inside it, because $c_0 + c_1 s$ is the plaintext together with that noise. An adversary that holds c_0 and c_1 subtracts and rounds, obtains $c_1 s$, and recovers s by one ring inversion. The server holds c_0 and c_1, and definition 1 corrupts the server together with $t - 1$ clients, so this view is inside the model rather than outside it. Resubmitting a query is also what the protocol permits, so the adversary is semi-honest.

The cost is the number of repetitions that drives the averaged error below half a unit in each of the 2^{15} coefficients, which is on the order of $81\sigma^2$ for a resampled part of standard deviation σ. At eight times the fresh-encryption noise and a quorum of ten that is about 5×10^5 repetitions, which is the same order as the query budgets section V-F already contemplates. A duplicate check is therefore a required control and not a supplementary one. Re-randomizing a query defeats the check but not the control, because a re-randomized query is a different ciphertext and yields one independent sample rather than a repetition, and independent samples are the regime the flooding is sized for. Sizing that flooding for q decryptions costs half a bit for each doubling of q, and the budget is refreshed by rekeying once it is spent [51].

Raising the smudging is the second control and it is free on every axis the deployment measures. The values this protocol decrypts are one index and one label, so the precision the answer needs is a handful of bits against a plaintext scale of 2^{45}. Moving the smudging from eight times the fresh-encryption noise to 2^{25} multiplies the required repetitions by roughly 2^{40} and still leaves twenty bits of the scale. It costs nothing in latency, nothing in traffic, and nothing in the accuracy of the served answers, because the label is an argmax and not a score. It does not meet the $2^{\lambda/2}$ rule, and we do not claim that it does. It removes the attack above at no price, which is a different and weaker statement, and it is the configuration we recommend.

One deviation the theorems do not cover belongs here as well. Collective key generation has each party publish a share of the public key, and the shares are summed. A party that publishes last, after seeing the others, can subtract their contribution and leave a public key whose secret it holds alone. Every ciphertext uploaded under that key is then readable by that party, and the deviation is detected only later, when the honest decryption shares fail to reconstruct, by which time the uploads have been read. The standard remedy is to commit the shares before opening them, which costs one round and is independent of the ring parameters. Theorem 2 lets a coalition deviate in the shares it contributes, and its proof disposes of that deviation for decryption shares, where a wrong share aborts the query. Key-generation shares are not covered by that argument, and a deployment that omits the commitment round is outside the theorem.

V. Experiments

Each subsection below states a claim in one sentence, describes the experiment that tests it, and reports the result. The claims are, in order, that a federated head over local representations is a usable model , that its accuracy is competitive with one-shot federated learning on that literature's own partition, that the federation can choose between arrangements without decrypting either, that the cryptographic layer is affordable, including the traffic that recurs, and that what a participant can learn is bounded. Section V-G then states what disclosing the model would have bought, and section V-H records what the protocol does not cover.

A. Setup

The trainable unit is a rank-8 low-rank adapter with its down-projection frozen at a shared public initialisation, together with a classifier head. We use no labelled public data anywhere.

We evaluate on four text classification tasks of increasing label-space size, AG-News with 4 classes, TREC with 6, DBpedia with 14 and Banking77 with 77, using a frozen RoBERTa-base encoder [28], and on CIFAR-100 with a frozen ViT-B/16 [29]. Each text task draws at most 20,000 training and 5,000 test examples under the seed, and each vision task at most 10,000 and 2,000. Only TREC and Banking77 fall below these caps, and we use those whole. Data are partitioned across clients by a Dirichlet distribution over labels with concentration α, the standard label-skew partition [52], [53], which induces label-distribution skew in the usual taxonomy [54] and holds the feature distribution fixed across clients. A smaller α leaves each client a few dominant classes and almost none of the rest. Unless stated otherwise there are $N = 10$ clients, $\alpha = 0.1$, the local trajectory is 200 steps, and every number is the mean over three seeds. Each client reserves at least one held-out example from every class it holds, as section III-D requires, and tops the held-out set up to one tenth of its shard at random.

We write A_{sel} for the accuracy of the servable arrangement that section V-D selects. Three reference points appear throughout, and none of them is servable under the threat model of section IV. We write A_{loc} for the accuracy a client obtains alone. We write A_{dis} for the accuracy of the disclosed model, in which the adapter and the head are both aggregated and decrypted. We write A_{pool} for the accuracy of one model trained on the union of the clients' data.

Two differences follow, and the paper reports both. The quantity $A_{\mathrm{dis}} - A_{\mathrm{sel}}$ is the price of never disclosing a model. The quantity $A_{\mathrm{pool}} - A_{\mathrm{dis}}$ is the price of the partition.

B. Utility of the Federated Head

Sharing only the classifier head, over representations that each client adapts locally, yields a model usable across the whole label space, and the preferable arrangement depends on the size of the label space.

Table II reports both servable arrangements and both reference points. The arrangement labelled *shared head* gives every client the same head over the bare public backbone. *Personal adapter* gives each client the shared head over its own locally adapted representation.

Three observations follow. First, the federation raises accuracy on every task. A client alone reaches 0.20 to 0.48, while the federated head reaches 0.61 to 0.79 on four of the five tasks. The gap is largest exactly where a client's own data covers least of the label space.

Second, the preferable arrangement changes with the size of the label space. On the small label spaces the shared head wins, 0.17 on AG-News and 0.18 on TREC. On the large ones the personal adapter wins. The shared head collapses to 0.206 while the personal adapter holds 0.686. The collapse is a coverage failure. With 77 classes each head row is decided by few clients or none.

Third, the two failure modes are complementary. The shared head fails when coverage is thin. The personal adapter fails

TABLE II

ACCURACY ON THE GLOBAL TEST SET, THREE SEEDS, $N = 10$, $\alpha = 0.1$. THE SERVABLE COLUMNS ARE THE TWO ARRANGEMENTS THE PROTOCOL ANSWERS QUERIES FROM. THE REFERENCE COLUMNS ARE NOT SERVABLE UNDER THE THREAT MODEL OF SECTION IV. THE ESTIMATOR SELECTS PER SEED, SO THE SELECTED ACCURACY IS 0.669 ON AG-NEWS, 0.756 ON CIFAR-100, AND THE BOLDED VALUE ELSEWHERE. WE DID NOT RUN THE POOLED REFERENCE ON CIFAR-100. ON ONE SEED THE DIRICHLET DRAW ASSIGNS FEWER THAN TWENTY SAMPLES TO SOME CLIENTS, WHICH CANNOT TRAIN, SO THE EFFECTIVE FEDERATION IS SEVEN ON AG-NEWS AND NINE ON TREC.

		servable			reference	
Task	C	shared head	adapter	alone	disclosed	pooled
AG-News	4	**0.649**	0.479	0.475	0.739	0.921
TREC	6	**0.607**	0.429	0.400	0.711	0.967
DBpedia	14	**0.789**	0.754	0.451	0.925	0.988
Banking77	77	0.206	**0.686**	0.249	0.760	0.923
CIFAR-100	100	0.748	**0.774**	0.197	0.784	—

TABLE III

SENSITIVITY ON DBPEDIA ALONG THE PROTOCOL AXES, EACH VARIED AROUND THE DEFAULT OF $N = 10$, $\alpha = 0.1$ AND 200 LOCAL STEPS. THE ROW REPORTS THE ACCURACY OF THE ARRANGEMENT THE ESTIMATOR SELECTS. THE SKEW ROW IS THE MEAN OVER THREE SEEDS. THE LOCAL-STEPS AND CLIENT-COUNT ROWS ARE A SINGLE SEED, AND THEIR DEFAULT CELLS ARE THAT SEED'S VALUE RATHER THAN THE THREE-SEED MEAN.

Label skew α				
	0.05	0.10	0.30	1.00
selected arrangement	0.814	0.789	0.915	0.970
selected	shared	shared	personal	personal
Clients N				
	10	20	50	
selected arrangement	0.803	0.826	0.882	
Local steps				
	100	200	400	
selected arrangement	0.710	0.803	0.871	

when each client's representation drifts far enough that a common head no longer transfers. Neither arrangement dominates the other, so the federation must select between them.

a) Sensitivity: Figure 4 varies the protocol axes around the default on DBpedia. Accuracy falls with skew, from 0.970 at $\alpha = 1.0$ to 0.789 at $\alpha = 0.1$. Longer local training helps. Accuracy rises with the size of the federation, from 0.803 at $N = 10$ to 0.882 at $N = 50$.

The more informative reading concerns which arrangement the estimator selects. The estimator chooses the shared head at $\alpha \in \{0.05, 0.1\}$ and the personal adapter at $\alpha \in \{0.3, 1.0\}$, on every seed in each case. The crossover is therefore not a property of the label space alone, as section V-B might suggest, but of coverage. The personal adapter is preferable once each client sees enough of the label space for its own representation to transfer, and the shared head is preferable when it does not. The estimator locates that crossover from encrypted local evidence, with no test set and no fitted threshold.

C. Comparison with One-Shot Federated Learning

The protocol gives up no accuracy for protecting the contributions, where the differentially private one-shot methods give up an amount that grows as their budget tightens. The peer group evaluates on different tasks from ours, so we ran our protocol on theirs. We adopt two published partitions of CIFAR-10: $N = 5$ with $\alpha \in \{0.1, 0.3\}$, the configuration of [16], and $N = 20$ with $\alpha \in \{0.04, 0.16\}$, the configuration of [19].

TABLE IV

CIFAR-10 ON TWO PUBLISHED PARTITIONS, THREE SEEDS, ON THE WHOLE 10,000-IMAGE TEST SET. THE COLUMNS ARE THE QUANTITIES DEFINED IN SECTION V-A, ALL MEASURED ON OUR PROTOCOL AT THE PARTITION EACH PRIOR PAPER PUBLISHED.

N	α	A_{loc}	A_{sel}	A_{dis}	$A_{\text{dis}} - A_{\text{sel}}$
5	0.10	0.384	0.949	0.963	0.013
5	0.30	0.570	0.952	0.966	0.015
20	0.04	0.212	0.950	0.947	−0.003
20	0.16	0.397	0.953	0.961	0.007

At $N = 5$ and $\alpha = 0.1$ we measure $A_{\text{sel}} = 0.949$, against the 0.503 of [16]. We do not present that margin as a result. Those methods train a ResNet-18 from scratch, whereas every client here fine-tunes a frozen public ViT-B/16. Absolute accuracy across papers therefore measures the backbone.

What compares across papers is the accuracy each method gives up for protecting the contribution, since every paper measures that against its own unprotected baseline on its own architecture, and the backbone cancels. Read that way the peer group falls into three cases. A method either leaves the contribution unprotected, protects it with differential privacy, or protects it cryptographically.

DENSE and FedDF report no such cost, because neither sets out to protect the contribution. What each client sends, a trained model in one case and repeated predictions over many rounds in the other, is available to the server in plaintext, and by the evidence of section VI-A those are the quantities an adversary would target.

FedAUXfdp does protect its contribution, and its cost is governed by the budget. Against its own undefended baseline of 0.752 on CIFAR-10 at $N = 20$ and $\alpha = 0.16$, the published figures give up 0.006 at $\varepsilon = 1.0$ and 0.029 at $\varepsilon = 0.5$, then 0.413 at $\varepsilon = 0.1$ and 0.626 at $\varepsilon = 0.01$ [19].

Our contribution is protected cryptographically, and that protection costs no accuracy, by construction, since the decrypted result matches the plaintext computation to the encoding precision of the scheme. Withholding the model is a separate charge, and no method above incurs it. Over table IV the charge lies between -0.003 and 0.015, and over table II between 0.03 and 0.14.

a) A partition at which the charge is negative: At $N = 20$ and $\alpha = 0.04$ we measure $A_{\text{sel}} > A_{\text{dis}}$, by 0.003 on average, on two seeds of three. The mechanism is the following. At that concentration each client holds close to a single class, so the aggregate of the adapters moves the representation toward every local task at once and improves none of them. The arrangement that leaves the public backbone unchanged avoids this, and it is the arrangement the estimator selects. We record one property of this cell: the Dirichlet draw at $\alpha = 0.04$ assigns fewer than twenty samples to some clients, which cannot train, so the effective federation size is 17 to 19 rather than 20.

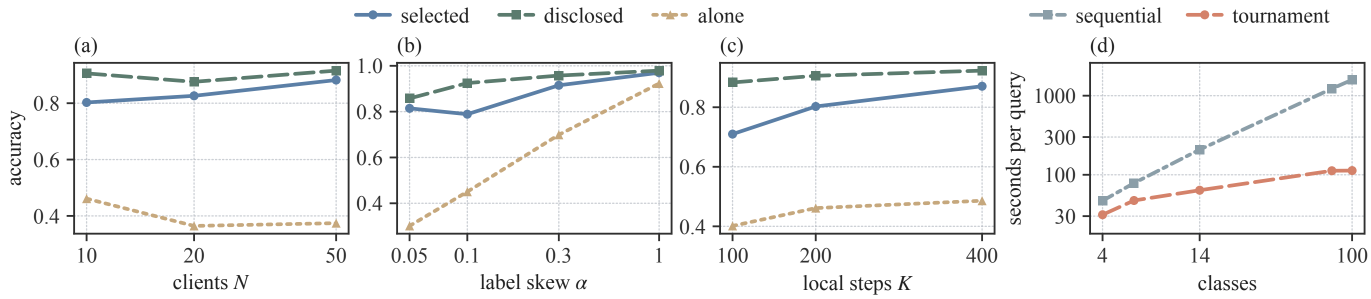

Fig. 4. The protocol along four axes. Panels (a) to (c) vary the federation size, the label skew and the local trajectory on DBpedia around $N = 10$, $\alpha = 0.1$ and 200 steps. Panel (d) gives the cost of one encrypted argmax against the label space, at $N = 10$ and ring degree 2^{15} under collective refresh, for the tournament and for the sequential fold it replaces. Panels (a) and (c) are one seed and panel (b) is the mean over three.

One difference bounds the federation's own contribution independently of the backbone, because we measure both of its terms on the same backbone. At $N = 5$ and $\alpha = 0.1$ we measure $A_{\text{loc}} = 0.384$ against $A_{\text{sel}} = 0.949$. That difference is what the protocol supplies.

b) Selection on these partitions: The estimator of section V-D selects correctly in 11 of the 12 cells behind table IV. The exception is $N = 5$ with $\alpha = 0.3$ on one seed, where the selected arrangement reaches 0.945 and the other reaches 0.953.

TABLE V
CHOOSING BETWEEN THE TWO SERVABLE ARRANGEMENTS WITHOUT DECRYPTING EITHER. CELLS ARE COUNTED OVER THE FIVE TASKS AND THE FOUR CIFAR-10 PARTITIONS, AT THREE SEEDS EACH. REGRET IS THE ACCURACY GIVEN UP BY A WRONG CHOICE, AVERAGED OVER SEEDS AND SUMMED OVER THE NINE SETTINGS.

Selection rule	correct cells	regret
majority of held-out accuracies	8/27	0.470
same, class-balanced within a client	11/27	0.454
global-prior estimator, zero fill	23/27	0.029
global-prior estimator, rare-class fill	**24/27**	**0.021**

D. Accuracy of the Selection Rule

The direct procedure for choosing between the two arrangements fails systematically, and the estimator of section III-D corrects it while disclosing only the index of the selected arrangement. We establish the failure first, since it is what motivates the estimator. Each client measures both arrangements on data held back from its own training and the federation takes the sample-weighted majority. Across all fifteen cells of table II, this procedure selects the personal adapter every time, including on AG-News and TREC where the personal adapter is 0.17 and 0.18 worse. Weighting each class equally within a client's held-out set does not help, and selects identically.

The failure is structural. The shared arrangement is one model, and the union of the clients' data is the whole training set, so pooled held-out measurements estimate its accuracy on the global distribution to within 0.02 on average. The personal arrangement is N different models, and client j's held-out data measures only client j's own model, on client j's own distribution, which is the distribution that model was fitted to. Its measured accuracy exceeds its true global accuracy by 0.28 on average. A client's held-out set also contains only the classes that client holds, so the very quantity that decides the comparison, accuracy on classes the client has never seen, the client cannot measure locally at all. The procedure compares an accurate number against an optimistic one, so it is not close to correct and it does not become correct with more held-out data.

Table V reports the estimator of eq. (2) against that baseline. Scoring both arrangements under the federation's own class prior, letting the shared arrangement pool evidence across clients and scoring the personal arrangement's unobserved classes at zero, selects correctly in 12 of 15 cells. Filling the unobserved classes from each client's rarest classes instead of with zero selects correctly in 13. Both are parameter-free in the sense that matters: the rule compares two numbers on the same absolute accuracy scale, with no fitted threshold.

Two properties of the estimator merit separate mention. The decisive case is Banking77, where choosing wrongly costs 0.48. The estimator selects correctly on all three seeds, by margins of 0.13 to 0.24. And the estimator responds to the data On the AG-News seed where the partition drops three clients and leaves the shared head at 0.402 and the personal adapter genuinely better, it switches to the personal adapter, which a rule keyed to the number of classes would not do.

The estimator yields a calibrated accuracy estimate, not only a ranking. Its estimate of the shared arrangement's global accuracy is within 0.019 on average across the fifteen cells of the five tasks and within 0.005 across the twelve CIFAR-10 cells, computed entirely from encrypted local evidence with no access to a test set.

The twelve CIFAR-10 cells separate the two rules more sharply than the five tasks do. The shared head over the bare backbone wins in eleven of them, and the majority of held-out accuracies chooses the personal adapter in all twelve, so it is correct once. The estimator is correct eleven times, and its one error costs 0.006. The failure of the vote is not a matter of close calls. A client measures the personal arrangement on its own data, where that arrangement is at its best, and the vote is the sum of twelve such measurements.

a) Disclosure of the estimator: One value is decrypted for the whole federation, the index of the arrangement adopted. The per-client per-class accuracies are not decrypted, and this matters. Together with the count matrix they would describe each client's label distribution alongside its model quality.

E. Cryptographic Cost

The cryptographic cost of the protocol is dominated by one-time setup, and the recurring per-query traffic is a few megabytes. We do not simulate the cryptography. We implement the protocol in real multiparty CKKS on Lattigo [55], [27]. All figures use ring degree 2^{14} for the shallow operations and a deeper chain at ring degree 2^{15} for the circuits that need it. Timings are single-run wall clock on one core of a VALAR CPU node. Communication figures are exact.

a) Cost of query encryption: Applying the head to an encrypted query costs $39.0\,\text{ms}$ against $5.4\,\text{ms}$ for a plaintext one, a factor of seven. That factor buys the property that the server never sees the feature vector and therefore cannot invert it to recover the client's input, and it is small beside the argmax that follows.

b) End-to-end query cost: On one core a query takes $400.8\,\text{s}$ at four classes and $1468.3\,\text{s}$ at a hundred. The arithmetic and the key switch account for $0.24\,\text{s}$ of that at every label-space size. Everything else is the encrypted argmax, and 97 per cent of the argmax is the server restoring levels, which it does 17 times at four classes and 62 times at a hundred. The tournament needs $\lceil \log_2 C \rceil$ rounds against the $C-1$ of the sequential fold it replaces (fig. 4).

Against the closest published system, slytHErin evaluates a twenty-layer network under the same multiparty arrangement in $245.58\,\text{s}$ at three parties and $354.17\,\text{s}$ at twenty, batched on twelve cores and returning the score vector [56], where our $400.8\,\text{s}$ and $1468.3\,\text{s}$ cover one unbatched query on one core and return only the label, which section V-F shows costs an adversary between 16 and 260 times more to recover. Their batch amortizes to about $0.95\,\text{s}$ per sample at ten parties, and their cost lies in the network where ours lies in the argmax, so the two figures size the setting rather than rank the systems. They return a score vector, so they stop before the reduction that is 97 per cent of our query. A system that returned a label would pay it.

Three levers apply to that figure, and they differ in what they move. Multi-core execution does not shorten the circuit, because the rounds are sequential by construction and the comparisons within one round are already packed into a single operation on a single ciphertext. It applies inside each primitive instead, across the residue moduli of the residue number system decomposition, and our single-threaded measurement uses none of it. Overlapping computation with communication raises throughput rather than lowering latency, because the level restorations lie on the critical path of one query, while the server that answers many queries can pipeline them. Hardware acceleration is the only lever that acts directly on the dominant term. The argmax is bootstrap-bound, which is where a GPU implementation helps most and, for the reason given below, where we decline to project a single number.

c) The encrypted reciprocal: Inverting the per-class totals under encryption, which section III-B requires so that no coalition can read the remaining client's class histogram,

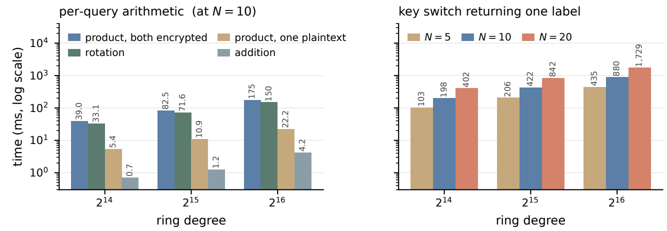

Fig. 5. Per-operation cost against the two axes that set it, from one measurement over the whole grid. Left: the per-query arithmetic at $N = 10$, which the ring degree sets and the federation size does not touch. Right: the key switch that returns one label, which grows with both. Read in seconds, the left panel spans 0.0007 to $0.18\,\text{s}$ and the right panel spans 0.10 to $1.7\,\text{s}$. Neither is the cost of a query. The encrypted argmax between them takes $400\,\text{s}$ at four classes and $1468\,\text{s}$ at a hundred, so one query totals $400.8\,\text{s}$ and $1468.3\,\text{s}$, of which the argmax is more than 99 per cent.

takes $4.21\,\text{s}$ at $N = 10$ and needs two collective refreshes. It agrees with the plaintext reciprocal to a relative error of 1.9×10^{-8}, so it costs no accuracy. The figure grows slowly with the federation, $3.16\,\text{s}$ at $N = 5$ and $6.30\,\text{s}$ at $N = 20$, because only the refreshes involve the clients. This is the one deep operation in building the shared head, and it is paid once per aggregation rather than per query, so a few seconds is immaterial beside the local training it follows.

d) The argmax: The argmax is the one operation in the protocol that is not shallow. Evaluated as a sequential fold of $C - 1$ comparisons it takes about $16\,\text{s}$ per comparison, which is roughly twenty-six minutes at a hundred classes. Packing the logits into one ciphertext and reducing them by a tournament lowers the number of sequential comparisons from $C - 1$ to $\lceil \log_2 C \rceil$ and brings the same computation to under two minutes at a hundred classes, a fourteenfold reduction. Both figures restore levels by collective refresh, so the ratio measures the reduction and not the mechanism. Because homomorphic evaluation under a collectively generated key is identical to evaluation under a single key, single-key results carry over, and GPU implementations of the same reduction report a full-vocabulary argmax in well under a second [57].

e) The two axes that set the cost: The ring degree is the security parameter, and the number of clients decides how many shares a decryption needs. Figure 5 separates them, from one measurement over the full grid.

The per-query arithmetic answers to the ring degree alone. It roughly doubles with each doubling of the degree, and it does not move with N: the encrypted product measures 38.4, 39.0 and $39.2\,\text{ms}$ at $N = 5$, 10 and 20.

The key switch that returns a label answers to both, and doubles with each. It runs from $103\,\text{ms}$ at the smallest setting to $1.73\,\text{s}$ at the largest. Traffic follows the same law, at $2.0\,\text{MiB}$ per participating client, because each contributes one independent share and the server only sums them. The encrypted reciprocal grows more slowly, $3.16\,\text{s}$ at $N = 5$ against $6.30\,\text{s}$ at $N = 20$, since only its two refreshes involve the clients.

f) Hardware acceleration: The figures above are single-threaded CPU, and the argmax is the only operation slow enough to matter, so the question is what specialised hardware does to it. Since the cost of a CKKS operation grows with the ring degree, the comparison is only meaningful at a fixed one. Figure 5 gives our figures, 39.0, 82.5 and 174.9 ms for a ciphertext-by-ciphertext product at ring degrees 2^{14}, 2^{15} and 2^{16}, and 33.1, 71.6 and 149.7 ms for a rotation. Phantom, a CUDA implementation of CKKS [58], reports the same primitives at ring degree 2^{16} on an A100 between 1.61 and 8.56 ms for the product and between 1.53 and 8.49 ms for the rotation, the range running over modulus chains of 29 to 41 levels. We give no single ratio against those figures. Ours are single-threaded Lattigo at a chain we did not match to theirs, and a cross-library ratio at unmatched depth is not a measurement.

The circuit is bootstrap-bound, so one number sets the accelerated figure. Level restoration is 97 per cent of a query and costs 23.1 s on one core at every label-space size, against the 28.6 s a full-slot bootstrap costs in OpenFHE at the same ring degree and 29 levels [59]. We measure one restoration under Phantom on an NVIDIA H100 at 2.28 s, at the ring degree 2^{16} the protocol restores at, a factor of ten. Phantom publishes no bootstrapping benchmark, and no GPU library we found restores levels under collectively generated keys, so this figure has no published counterpart. Substituting it, and leaving the local evaluation on the processor, gives 48.9 s at four classes, 100.1 s at fourteen and 176.6 s at a hundred, against the 400.8 s and 1468.3 s of table VII. The local evaluation, which we did not port, is then the larger half of a query above four classes.

g) One query, end to end, on a real head: Every figure above runs on synthetic vectors, so none of them says the encrypted path answers a real query the way the plaintext model does. We ran that comparison. On the recorded AG-News head at seed 42, ten parties, ring degree 2^{15} and scale 2^{45}, thirty-two encrypted answers over the two servable arrangements all equal the plaintext answers. The hardest case has a top-1 to top-2 gap of 0.037, which the scaling takes to 0.0068, and its decoded index sits within 3×10^{-8} of the integer. Nothing failed, so the resolution limit is bounded below by that gap and not above.

Two of the sixteen answers under the shared arrangement are queries the head itself gets wrong, and the encrypted path reproduces those errors. That is what a faithful serving path does. It agrees with the plaintext computation, not with the ground truth.

h) Where this sits against published work: No published system takes an argmax under encryption in a multiparty scheme, so there is no matched number to compare with. The nearest match on the primitive is NEXUS, which computes the same sign-based argmax under CKKS at the same ring degree 2^{16} and returns an encrypted one-hot vector, at 54 s over a label space of 30,522 [57]. Three conditions separate that figure from ours. It is single-key, so it runs no collective key switching and no smudging. It uses 32 threads where we use one core. And it is amortized over a batch of 32 inputs, where we serve one query.

The multiparty systems that exist do not take the argmax. POSEIDON answers a query in 0.38 s at ten parties, at ring degree 2^{13} with six levels, and returns the prediction vector after key-switching it to the querier [22]. slytHErin returns scores in 245.58 s for a twenty-layer network at three model-providers [56]. Two ring-degree doublings and the argmax itself separate the first from our figure, and the second evaluates a whole network where we evaluate one linear map and then compare its outputs. The cost we report is the cost of refusing to release the scores.

TABLE VI
COMMUNICATION AT $N = 10$, ON THE FIFTEEN-MODULUS CHAIN AT RING DEGREE 2^{15} THAT THE ENCRYPTED ARGMAX REQUIRES. SHARE SIZES DO NOT DEPEND ON THE NUMBER OF CLIENTS. THE PER-QUERY TOTAL DOES, BECAUSE THE QUORUM RETURNS ONE KEY-SWITCHING SHARE EACH, AND IT DOES NOT DEPEND ON THE NUMBER OF CLASSES, BECAUSE THE LEVEL RESTORATIONS ARE LOCAL TO THE SERVER.

When	Item	Per client
setup, once	collective public key share	4.25 MiB
	relinearization key share	102.0 MiB
	rotation keys, seven of them	238.0 MiB
training, once	encrypted head displacements	12 to 44 MiB
per query	encrypted features, uploaded	7.5 MiB
	encrypted label, returned	1.0 MiB
	key-switching shares, ten	5.0 MiB
	total per query	**13.5 MiB**

i) Communication: The protocol is one-shot in the sense of section I. No intermediate training artifact is ever exposed. It is not silent. Key generation precedes it, selection costs at most $2NC$ encrypted comparisons, once, and serving costs traffic per query.

We measured what that selection actually costs, because the comparison count alone does not say. At five clients and four classes it takes 400 s and moves 3.4 GiB. At twenty clients and a hundred classes it takes 7111 s and moves 145.4 GiB. The federation pays this once, before it answers anything, and no part of it recurs per query.

The bound holds and the cost sits elsewhere. Our implementation evaluates 321 sequential sign circuits at twenty clients and a hundred classes, where $2NC$ allows 4000 comparisons. What the count does not carry is depth. Each comparison under CKKS is a polynomial approximation of the sign, every such approximation consumes levels, and restoring them is 99.8 per cent of the traffic above. This is the price the current comparison literature charges [60], [61], and we pay it rather than decrypt the two scores.

The selection is correct at every point we measured. It decodes the winning arrangement in all twelve cells of the grid, and the per-class totals it combines come back with a relative error near 10^{-8}. Section V-D reports which arrangement the rule chooses and how often that choice is right. This paragraph reports only that the encrypted evaluation reproduces it.

The training upload is the one row that moves with the number of classes. A head displacement is C rows of 768 values and a bias, and a ciphertext at ring degree 2^{15} carries 16,384 slots, so a client sends $\lceil 769C/16{,}384 \rceil$ ciphertexts for each of the two arrangements and one further ciphertext for its per-class counts. On the eight-modulus chain the aggregation needs, a ciphertext is 4.0 MiB. That is three ciphertexts and 12 MiB for AG-News, TREC, CIFAR-10 and DBpedia, nine and 36 MiB for Banking77, and eleven and 44 MiB for CIFAR-

100. It is paid once and it is smaller than the rotation keys of the setup in every case.

The per-query total is $8.5 + 0.5N$ MiB, since the quorum returns one key-switching share each. It is 11.0 MiB at $N = 5$ and 18.5 MiB at $N = 20$, and it is 6.8 MiB at ring degree 2^{14} and 27.0 MiB at 2^{16}. It does not vary with the number of classes. Setup costs 344 MiB per client, once.

j) How this scales in the federation: Every per-party quantity in table VI is independent of the client count. We measured the grid at $N \in \{5, 10, 20\}$, at ring degrees 2^{14}, 2^{15} and 2^{16}, and on both the aggregation and the serving chain, and the collective public key share, the relinearization key share, the rotation key share, the ciphertext sizes and the key-switching share are the same at every client count. This is what the structure of the scheme predicts, since a party's share is a function of the ring and the modulus chain and not of how many parties there are, and the measurement confirms it rather than discovering it.

Two consequences set the cost of a large federation. What a client sends does not grow as clients are added, so setup stays at 344 MiB for each of them and the federation's total key-generation traffic grows linearly in N, which is 3.4 GiB at $N = 10$ and 17 GiB at $N = 50$. A query costs the querier a fixed 8.5 MiB and the quorum one key-switching share each, so per-query traffic grows linearly in t and not in N. A deployment that lowers the threshold for availability therefore also lowers the traffic on every query, and section IV-F states what it costs in privacy, which is a shorter distance to the same cap rather than a higher cap.

One design decision governs the recurring figure. The argmax spends its levels and must have them restored several times per query. Restoring them by a collective refresh requires a share from every client for every refresh, which at a hundred classes amounts to roughly 1.6 GiB of traffic for a single label. Generating bootstrapping keys once, collectively, and letting the server restore levels on its own reduces that traffic to zero, because evaluation under a collectively generated key is identical to evaluation under a single key. The protocol specifies the latter. We have now measured both, and the trade is not one-sided.

Table VII reports the specified design beside the one the earlier figures were taken on. Restoring levels at the server costs about thirteen times the latency at every label-space size, from 400.8 s against 31.2 s at four classes to 1468.3 s against 113.0 s at a hundred. The bootstrapping keys cost 8.70 GiB, generated collectively in 46 s, against a stored figure of 15.5 MiB that measured the extended secret key rather than the key set the server evaluates with. The saving is real and it is in traffic. One key generation of 8.70 GiB removes about 1.6 GiB from every subsequent query, so the two designs break even near the sixth query and the specified one wins after it. Precision moves as well. The collective refresh reproduces the plaintext maximum exactly, and the server-side bootstrap agrees to 7.3×10^{-5}.

k) Which mechanism produced the reported latency: Every latency above is the specified design, in which the server restores levels on its own under the collectively generated keys. Table VII sets it beside the collective refresh, which is about thirteen times faster and costs a share from every client at every restoration. We report the specified design because it is the one the method describes, and because a protocol that moved gigabytes to the clients on every query would not be servable at the federation sizes this paper considers. Every figure is single-core, and the restoration is the part that would gain most from threads.

TABLE VII
THE TWO LEVEL-RESTORATION MECHANISMS AT $N = 10$, EVALUATION AT RING DEGREE 2^{15}. THE SPECIFIED DESIGN RESTORES LEVELS AT THE SERVER UNDER COLLECTIVELY GENERATED BOOTSTRAPPING KEYS AT RING DEGREE 2^{16}. THE MEASURED ALTERNATIVE RESTORES THEM BY A COLLECTIVE REFRESH, WHICH COSTS A SHARE FROM EVERY CLIENT. LATENCY IS SINGLE-RUN WALL CLOCK ON ONE CORE OF A VALAR CPU NODE.

C	4	6	14	77	100
collective refresh, s	31.2	47.3	63.7	112.4	113.0
server bootstrap, s	400.8	612.5	832.4	1473.0	1468.3
bootstraps	17	26	35	62	62
error, server bootstrap	1.3 to 7.3×10^{-5}, against exact for the refresh				

l) Correctness: The encrypted argmax agrees with the plaintext maximum to 7.3×10^{-5}, which is far below the gap between adjacent logits, so it selects the same class. The accuracy figures of table II are therefore unaffected by the cryptography.

F. Residual Leakage

The protocol exposes no training-time artifact and no model, so the only remaining channel is the sequence of answers the served model returns, and that channel is bounded by the query allowance rather than eliminated. The training-time surface is removed by construction. No gradient, update, or data-derived summary is ever available in plaintext, there is one exchange rather than many, and its single artifact is a ciphertext. The model is removed as well: no party holds it in plaintext at any point, so the white-box attacks that a released model admits have no object to attack. What remains is a client that can ask the served model questions and read labels.

Two distinct bounds apply to this channel, and they should not be conflated. The first is cryptographic and is stated in theorem 1: the protocol's messages are computationally independent of the private data. The second is not cryptographic. A client that queries the served model at points of its own choosing can, with enough queries, reconstruct a functional copy of the shared head. Returning only the label rather than the logits denies the linear solve that would otherwise recover the head in as many queries as the feature dimension, which raises the cost but does not make it infinite. We therefore state the guarantee as a bound on queries, and we measure what that bound has to be.

a) Cost of extraction: We give the adversary every advantage the protocol admits. It is a participating client, so it computes query features itself and may submit an arbitrary vector rather than a real input, and it is charged one query per label. It fits a linear model to the labelled queries and we score the copy by *fidelity*, the rate at which it agrees with the served model on held-out features. Table VIII reports the result.

TABLE VIII
LABEL-ONLY EXTRACTION OF THE SERVED HEAD. MEAN FIDELITY OVER THREE SEEDS AND BOTH ARRANGEMENTS, AGAINST THE NUMBER OF QUERIES SPENT. THE MAJORITY COLUMN IS THE SHARE OF THE MOST COMMON LABEL, WHICH IS WHAT A COPY ACHIEVES FOR FREE.

Task	C	majority	10^3	5×10^3	2×10^4	5×10^4	2×10^5
AG-News	4	0.488	0.635	0.823	0.936	0.973	0.993
DBpedia	14	0.250	0.387	0.612	0.804	0.901	0.971
Banking77	77	0.083	0.174	0.355	0.592	0.755	0.900

Extraction is possible and its cost is orderly. Reaching fidelity 0.90 takes about 1.2×10^4 queries on AG-News, 5.0×10^4 on DBpedia and 2.0×10^5 on Banking77. Those figures track the size of the head rather than the task. The head has Cd parameters, and in each case fidelity 0.90 costs between three and five times that number of queries, with 0.80 costing roughly 1.5 times and 0.95 roughly ten times. The multiplier is a function of the target, not a constant. Writing ε for the shortfall from a perfect copy, our three tasks fit $Q \approx 0.4\,Cd/\varepsilon$ across the measured range. A deployment can therefore set its allowance from C, d and the fidelity it is willing to concede, without repeating this experiment, and an allowance set for one target does not carry to another. A published measurement of the same interface agrees. Extracting a softmax model from class labels alone costs 100 queries per parameter for agreement above 99.9 per cent [32], and the curve here reaches 0.993 at 65 queries per parameter on AG-News.

The allowance follows. A coalition of $t-1$ clients, the largest that cannot decrypt, commands $(t-1)Q$ queries, so holding it below fidelity 0.90 at $N = 10$ requires Q below roughly 1.3×10^3 per client on AG-News and 2.2×10^4 on Banking77. The binding case is the small label space. A head with few rows is cheap to copy. It is also the case in which the shared arrangement is preferable (table II), so the deployments that gain most from the federation are the ones that must meter queries most tightly.

Fidelity measures agreement with the served model, and nothing more. A copy at fidelity 0.90 reproduces nine in ten of the served model's decisions, including its mistakes. It does not follow that the adversary has learned any client's data. The recovered object is a linear map over a public feature space, fitted from labels the adversary requested at points it chose itself, and no client record enters the attack at any stage. Turning a copy into a statement about a particular record is the separate problem of membership inference or reconstruction. That problem needs a surface this protocol does not expose, since no gradient, update or data-derived summary is ever available in plaintext. Section IV-F1 measures it and finds it at chance.

b) A second query strategy, and what it does not settle: We also ran a variant that spends half its budget bisecting toward the decision boundary and then fits the same linear model. It loses in 13 of the 15 cells, reaching 0.675 against 0.755 on Banking77 at 5×10^4 queries, because a bisection chain returns many nearly identical points and a linear fit gains less from them than from independent ones. Tramer et al. report the same ordering for multiclass models and offer the same reason, that the searches split across several decision boundaries [32].

TABLE IX
RANDOMISED RESPONSE ON THE RETURNED LABEL. COPY FIDELITY AFTER 10^5 LABEL QUERIES, AGAINST THE ACCURACY THE SERVED MODEL RETAINS, AT EACH BUDGET ε. MEAN OVER THREE SEEDS. THE MAJORITY SHARE A CONSTANT PREDICTOR ACHIEVES IS 0.488 ON AG-NEWS, 0.250 ON DBPEDIA AND 0.083 ON BANKING77.

Task		∞	8	4	2	1	0.5
AG-News	fidelity	0.989	0.986	0.954	0.889	0.774	0.598
	accuracy	0.649	0.648	0.620	0.494	0.372	0.303
DBpedia	fidelity	0.956	0.946	0.831	0.571	0.268	0.139
	accuracy	0.789	0.785	0.640	0.297	0.150	0.103
Banking77	fidelity	0.777	0.722	0.364	0.052	0.020	0.016
	accuracy	0.206	0.203	0.092	0.028	0.018	0.015

The ordering reverses once the budget is large against the head. On AG-News the variant crosses over at about 16 queries per parameter and at 65 it leaves 0.0041 of the decisions wrong against the random strategy's 0.0072, a factor of 1.77. Banking77 never reaches 3.4 queries per parameter at our largest budget, so its cells are measured below the crossover and the count of 13 reflects the budget grid rather than the strategy. We report the random strategy because it is the stronger one over the range an allowance would sit in, and because it is the simpler.

Neither strategy is the attack a cryptanalyst would mount. Both fit a model to labelled points. The published constructions instead solve for each decision hyperplane from points found on it, and tie the hyperplanes to one another so that the scale the argmax discards is fixed once [62], [35]. That route costs a factor logarithmic in the precision demanded rather than linear in it, so our figures bound the allowance from the wrong side and should be read as what a straightforward adversary pays.

c) Noise on the returned label: The query allowance is one way to price extraction. Perturbing the answer is the other, and it is what a reader who knows differential privacy will ask for first. We measure it. The server flips the returned label under randomised response at budget ε, which is the label-level mechanism [63], and we record both what the copy reaches and what the served model still scores. Table IX reports the result.

The mechanism does not separate the two quantities. At $\varepsilon = 8$ it moves neither, costing AG-News 0.001 of accuracy and the copy 0.003 of fidelity. Below that both fall together. On AG-News the budget that first costs the copy a fifth of its fidelity is $\varepsilon = 1$, where the copy still reaches 0.774 and the served model has fallen to 0.372, under the 0.488 share of its most common class. On DBpedia accuracy falls by a larger fraction than fidelity at every budget below $\varepsilon = 8$. On Banking77 the copy does degrade faster than the task below $\varepsilon = 4$, and only after accuracy has reached 0.028 against a majority share of 0.083, so the model the noise protects is already worse than a constant predictor.

This is the argument for metering queries rather than perturbing answers. Noise on the label is charged to every honest query and to every extraction query at the same rate, and the extraction campaign spends 10^5 queries where an honest client spends one. The allowance charges the two differently, which is the property the mechanism lacks. We therefore state the operational bound as a bound on queries, and we do not add noise to the answers.

The published defences do not offer a third option, and the reason is the encryption. Query-distribution monitoring reads the query and tests whether a client's stream looks like a sweep [64]. Adaptive misinformation reads the query and answers differently when it looks out of distribution [65]. Proof of work reads the query to price it [66]. Watermarking needs the answer to be a deterministic keyed function of the query, so that a repeat cannot be filtered out [67]. Our server holds a ciphertext of a feature vector and, by the same assumption that protects the head, learns nothing from it. Every one of these defences is a decryption in disguise, which is the argument of proposition 1 applied to a different gate.

Output randomisation is weak by a theorem as well as by our table. Against a server that answers incorrectly with constant probability ρ below one half, extraction costs a factor of $8/(1-2\rho)^2$ times a logarithm and no more [68]. That bounds what the defender buys. Table IX supplies the half their analysis leaves out, which is what the defender pays. One mechanism does survive both restrictions and we have not measured it. Drawing a fresh head from a distribution for each query, rather than perturbing the label, needs no access to the query and flips only answers near a boundary [69]. It has a known break at a higher query cost, and its accuracy cost under our arrangement is open.

d) Comparison with a released model: The contrast is not a matter of degree. A participant holding the model in plaintext has fidelity 1 at zero queries, may read the parameters directly rather than infer them, and is subject to no allowance because there is nothing to meter. Granting logits rather than labels would be nearly as generous. The head is linear in features the client computes itself, so $d+1$ logit queries recover it exactly by linear solve, which is 769 queries here against the 1.2×10^4 to 2.0×10^5 that labels demand. We confirm this directly, recovering the head to fidelity 1.000 from 769 answers when the served interface returns a distribution instead of an index. Restricting the answer to the label is therefore what converts extraction from a solve into a search, and the query allowance is what prices that search. The same conclusion has been reached from the other direction in deployment, where recovering a production model's final projection layer was made materially harder by withdrawing exactly the richer outputs this protocol never offers [33].

G. Comparison with Model Disclosure

Never disclosing the model costs accuracy, and table II allows the price to be read directly. The quantity $A_{\text{dis}} - A_{\text{sel}}$ is 0.071 on AG-News, 0.104 on TREC, 0.136 on DBpedia, 0.075 on Banking77 and 0.029 on CIFAR-100. The price is largest in the middle of the label-space range and smallest at the extremes, and it is paid for a property that no accuracy figure can substitute for. After the protocol ends, there is no plaintext model anywhere.

The last column of table II isolates the second difference of section V-A. On three of the four text tasks $A_{\text{pool}} - A_{\text{dis}}$ exceeds $A_{\text{dis}} - A_{\text{sel}}$. It is 0.182 against 0.071 on AG-News, 0.257 against 0.104 on TREC, and 0.163 against 0.075 on Banking77. DBpedia is the exception, at 0.063 against 0.136. On those three tasks most of the distance between a servable arrangement and a centralised model is therefore attributable to the partition of the data, and not to the decision never to decrypt.

We do not present the disclosed model as an alternative configuration of this protocol, since it does not satisfy the requirements of section III-A. Disclosing the model changes the threat model. A participant that holds the model in plaintext can inspect it, run inference on it without limit, redistribute it, and mount the white-box attacks that section VI-A surveys, and the query allowance of section V-F becomes meaningless. Those are different guarantees and they answer a different question. A deployment that can disclose the model should use a protocol designed for that, and prior one-shot federated learning provides several. The construction presented here addresses deployments in which disclosure is not permitted.

H. Scope and Limitations

a) Linearity of the shared quantity: This is what section III-B shows the never-disclosed requirement forces, and it is a real restriction. A shared adapter inside the backbone would adapt the representation for every participant rather than only for its owner. Serving such an adapter reduces to secure transformer inference (section VI-E) and composes with that literature at its cost.

b) A public constant in the serving path: The serving circuit compares logits through a polynomial approximation of the sign, and that approximation resolves a fixed absolute margin. The argmax does not change when every logit is multiplied by a positive constant, but the margin the circuit must resolve does. Our runs fix one such constant from the exported head. A deployment cannot do that, because the head is a ciphertext, so it needs a public bound on the head's norm instead. Section IV does not treat that bound as secret, and we do not measure what a loose one costs.

c) Bounds on model extraction: The query allowance of section V-F is an operational control. Where honest use is intrinsically high-volume, so that no allowance separates it from extraction, calibrated noise added to the returned labels bounds cumulative leakage instead [63]. That mechanism composes with the protocol without modification, and it is orthogonal to it. The protocol protects the contributions cryptographically, and differential privacy would protect the answers statistically. Every accuracy figure in this paper assumes it is not in use.

d) Malicious participants: The cryptographic guarantees assume semi-honest parties. A client that uploads a crafted head displacement can bias the shared head, and because contributions are encrypted the server cannot inspect them. Two directions are compatible with the encryption and are left to future work, an upload-time norm bound enforced by a zero-knowledge proof, and robust aggregation evaluated homomorphically.

e) Availability: Section IV states the protocol for a t-out-of-N threshold, and every measurement here is taken at $t = N$. That setting gives the strongest collusion resistance the structure admits and the weakest availability, since every client must take part for a label to be produced. Lowering t relaxes that at two costs. It admits coalitions of size t, and it tightens the query allowance, because the bound of section V-F scales with the largest coalition that cannot decrypt. Which point on that trade-off a deployment wants is a deployment question. We did not measure any point other than $t = N$.

The liveness cost of that choice should be stated plainly. The scheme admits any t with $2 \leq t \leq N$, and at a general threshold a query needs t clients rather than all of them, which is what makes serving survive an offline participant. Our implementation sets $t = N$, so in the configuration we measured a single unavailable client blocks every query until it returns. A deployment that needs serving to continue through churn sets $t < N$ and accepts coalitions of that size.

Section IV-F makes the price of that choice smaller than it appears. Lowering t raises the coalition's total allowance $(t-1)Q$, so a coalition reaches a given fidelity sooner. It does not raise the ceiling δ_{wb} on what a coalition can learn about an honest client, since that ceiling is a property of the shared head. The threshold therefore trades availability against the cost of an attack rather than against what the attack can ultimately achieve, and a deployment can buy liveness with a proportionate reduction in Q.

VI. Related Work

A. Attacks at Training Time and at Inference Time

Federated learning (FL) began as an iterative procedure in which a server repeatedly broadcasts a model, clients train it on their own data, and the server averages the returned updates [10]. In every round each client sends a new update computed on its private data, and several attacks have shown that these updates leak more than anything else the protocol reveals. A gradient allows reconstruction of the batch that produced it [47], [48]. An observer of the per-round updates infers membership and unintended properties of another client's data, and the repetition of those updates is what makes the attack strong [11], [12], [70]. A server that chooses the weights it broadcasts does not need inference at all, since crafted weights make clients return verbatim copies of their own inputs, even through a secure-aggregation layer [71], [72]. Several works share predictions instead of gradients, and the predictions leak too [73], [74], [75], [76].

Attacks that see only the final model are consistently weaker, and the field survey draws that separation explicitly [54]. Several works infer membership from a released model, or invert it from released confidences [77], [78], [79], [80], [81], and against models that generalise well they identify few members, and only at low false-positive rates [42]. The reason for the gap is simple. An update is a detailed view of one client's data and arrives every round, where the final model is a single average that generalisation moves away from any one training example. A protocol that never sends an update, and that sends anything only once, therefore leaves every adversary with the weaker attack.

B. Protecting Training with Cryptography

Secure aggregation lets the server learn only the sum of the clients' updates, using masks that cancel in the aggregate [82]. It is lossless, but it protects one round's sum inside an iterative protocol, and Kerkouche et al. and So et al. show that the revealed per-round sums themselves leak across rounds [83], [84]. Homomorphic encryption goes further by computing on contributions that stay encrypted. POSEIDON runs encrypted stochastic gradient descent under multiparty CKKS, with activations replaced by polynomial approximations so that the server can evaluate them homomorphically [22]. The guarantee is strong and the model loses no accuracy, but the approach is iterative and encrypted nonlinearity costs a deep multiplicative circuit with repeated bootstrapping. Several works measure how delicate those polynomial activations are [85], [86], [87], [88], [89]. That cost scales with the network being trained, so adapting a pretrained transformer by this route is out of reach.

Several works encrypt only the aggregation step of an otherwise plaintext training loop [90], [91], [92], [93], [94]. All of them remain bound to the iterative protocol, so the encryption cost is paid on every round and what is protected is a per-round sum rather than a complete contribution. Single-key homomorphic encryption forces the assumption that the server and the clients do not collude [93]. The threshold structure of section II-A removes that assumption.

Transfer learning itself has been placed under encryption. Several works train a classifier head, a linear or logistic model, or a low-rank adapter on frozen features under CKKS [95], [96], [97], [98], [99], [100], [101], [102]. These share our structural niche, a frozen public backbone with a small trainable unit, but they run an optimiser on ciphertexts, so multiplicative depth grows with the step count and every nonlinearity must be approximated. Several of them are also single-party, one data owner outsourcing computation rather than a collaboration of mutually distrustful parties. Our parties train on their own and must protect what they trained.

C. One-Shot Federated Learning and Differential Privacy

A parallel line collapses the many rounds into a single exchange [14], [15]. Several works exchange predictions on a shared public set, which in practice repeats the exchange [103], [104], [105]. Genuinely single-round methods distil the clients' models through a generator or synthesised inputs, or fuse the models directly, in one round or in stages the authors account as one [16], [17], [106], [107], [18]. These establish that one round can produce a usable model, but they work in plaintext, and each client sends an entire model or a synthetic distillate of its data. By the evidence of section VI-A that is what an adversary would target. A recent method sends no model at all, having each client run a frozen pretrained encoder and upload per-class sufficient statistics once [108]. Those statistics travel in plaintext, and per-class totals are exactly what section III-B keeps encrypted.

The one-shot methods that do address privacy use differential privacy. FedAUXfdp releases client contributions under a differentially private mechanism after distillation through a pretrained extractor [19], and related approaches protect a diffusion-generated surrogate or a teacher ensemble [21], [20], or apply a noise-free mechanism to the distillation itself [109]. These are our natural quantitative peers, and their privacy is lossy, because the noise a meaningful budget requires is added to the very artifact that is released. Differential privacy proves a statistical bound on how much a released artifact can reveal and buys that bound with accuracy. Encryption makes a contribution computationally indistinguishable from random to every party without the key and costs the model nothing. Several works apply the same trade-off to transfer learning, running noisy gradient descent on frozen features or on a low-rank adapter [110], [111], [112], [113], [114], [115]. They share our backbone-and-adapter structure, but they pay a budget-dependent accuracy tax on the model they release, and they are centralised or multi-round.

D. Federated Fine-Tuning and Task Arithmetic

A recent line federates parameter-efficient fine-tuning, communicating a low-rank adapter rather than a model. FedIT averages the adapters round by round [116], but per-factor averaging is biased, because a client learns a product of two factors and averaging the factors separately does not give the average of the products. Several works correct that bias by stacking factors, by reconstructing full products, by reconciling heterogeneous ranks, or by sharing one factor only [117], [118], [119], [120]. FFA-LoRA instead freezes the randomly initialised down-projection and trains only the up-projection, so that averaging the trained factor is exact [121]. Combining displacements from a shared initialiser is task-vector merging, which is equivalent to one-shot federated averaging, and one round of fine-tuning suffices for foundation models in plaintext [122], [123], [124]. None of this line protects the contributions, which is our subject.

E. Secure Inference

Answering queries from a model that stays encrypted is the subject of secure inference. Several works evaluate a transformer under encryption, either non-interactively under CKKS or between two or three parties with secret sharing, and in each case the nonlinearities of attention dominate the cost [57], [125], [126], [127]. This literature is what makes the disclosure setting of section III-C affordable, and we compose with it. The map served here is linear in the features, so serving a trained map that sits inside the backbone instead would reduce to exactly the problem these systems address.

Two systems sit closer. slytHErin evaluates a network under CKKS with the model held under a multiparty key, and it key-switches the prediction to the querier, which is the serving setting of section III-C [56]. The same group built the encrypted training scheme of section VI-B, so the pair is the closest existing route to a model that no party decrypts. It evaluates the whole network homomorphically, which is why the models it reports are a twenty-layer and a fifty-layer network on a handwritten-digit task rather than a pretrained encoder. And it returns the prediction vector, so the querier receives scores, which section V-F shows is a materially cheaper target for extraction than a label is.

TABLE X
THE CLOSEST SYSTEMS ON THE FOUR AXES THIS PAPER TURNS ON, AS EACH PAPER REPORTS THEM. THE PLAINTEXT COLUMN SAYS WHETHER ANY PARTY ENDS UP HOLDING THE TRAINED MODEL IN PLAINTEXT, AND *provider* MEANS ONE PARTY DOES WHILE THE QUERIER DOES NOT. SLYTHERIN AND CRYPTPEFT SERVE A MODEL THEY DO NOT TRAIN, SO THEIR ROUNDS CELL IS EMPTY. POSEIDON KEEPS THE MODEL UNDER THE COLLECTIVE KEY AND STATES THAT THE QUERIER DOES NOT OBTAIN THE MODEL'S WEIGHTS.

System	Rounds	Protection	Model in plaintext	Querier receives
DENSE [16]	one	none	yes	the model
GH-OFL [108]	one	none	yes	the model
FedAUXfdp [19]	one	DP	yes	the model
POSEIDON [22]	many	HE	no	the prediction
slytHErin [56]		HE	no	scores
CryptPEFT [128]	many	2PC	provider	scores
HE-OFT	one	HE	no	label

CryptPEFT divides the work as we do and reaches the opposite conclusion about who holds the secret. A public frozen backbone runs in plaintext on the client, the client encrypts the resulting features, only the adapter is evaluated under encryption, and the client receives the adapter's output [128]. Confining encrypted computation to a small unit after the representation is the motivation for our design as well. It is a two-party protocol, the trained unit is the provider's plaintext, and there is no training protocol, so it establishes that the serving arrangement is practical without addressing how mutually distrustful parties would build the unit at all. It reports one inference in 2.26 s on CIFAR-100. It is two-party and the adapter is one party's plaintext, so it evaluates no ciphertext-by-ciphertext product and restores no levels.

Constructions for the encrypted maximum faster than the tournament of section III-C exist, both for ranking and order statistics at constant comparison depth and for decoding a language model without decrypting the scores [129], [130]. Both are single-key and neither carries the collective refreshes that the threshold setting requires, so neither transfers unchanged.

Positioning. Table X places the closest systems on the axes that separate them. The multi-round cryptographic line protects every round but pays on every round, and what it protects is a per-round sum. The one-shot line exposes its single contribution in plaintext, or noises it and pays in accuracy. The fine-tuning line communicates a small object but protects nothing. A single prior scheme is at once one-shot, federated and encrypted, but only for a single-layer closed-form learner that cannot adapt a deep representation [131]. What remains unaddressed is a protocol that exposes no training-time quantity, that exchanges anything at all only once, and that never discloses the model even to the parties that built it. Section III works out what that forces, and the rest of the paper measures what it costs.

VII. Conclusion

We presented HE-OFT, a federated fine-tuning protocol in which no party receives the trained result. The design follows from the observation that federated learning's privacy risk is concentrated in the quantities a protocol exposes while the model is being built, and from the further requirement that in some deployments the finished model may not be distributed either. Each client fine-tunes on a frozen public backbone, retains its adapter locally, and uploads a single encrypted head displacement. The server combines those displacements under multiparty CKKS and never decrypts the result. A quorum of clients returns only the predicted label, addressed to the party that asked.

Withholding the model constrains what may be shared. Since the querying party must form its query from components it can evaluate itself, those components must be public, and the shared trained quantity must therefore attach after the last nonlinearity of the network. Across four text classification tasks and one vision task, the protocol produces a model stronger than any client obtains alone, at a cost of 0.03 to 0.14 accuracy relative to a model that is aggregated and disclosed.

Two directions follow. Robustness to actively malicious participants is outside the present guarantee, and reconciling robust aggregation with an encrypted contribution remains open. Serving a shared quantity that sits inside the backbone, rather than after it, reduces to secure transformer inference.

Acknowledgments

We acknowledge the support of TÜBİTAK (the Scientific and Technological Research Council of Türkiye) projects 124E091 and 124N941. The authors used Claude to enhance the manuscript by shortening sentences, correcting typos, and improving grammar.

References

[1] R. Bommasani, D. A. Hudson, E. Adeli *et al.*, "On the opportunities and risks of foundation models," *arXiv preprint arXiv:2108.07258*, 2021.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *ICLR*, 2022.

[3] M. J. Sheller, B. Edwards, G. A. Reina, J. Martin, S. Pati, A. Kotrotsou, M. Milchenko, W. Xu, D. Marcus, R. R. Colen, and S. Bakas, "Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data," *Scientific Reports*, vol. 10, p. 12598, 2020.

[4] European Parliament and Council of the European Union, "Regulation (eu) 2016/679 (general data protection regulation)," Official Journal of the European Union, L 119, pp. 1–88, 2016.

[5] U.S. Department of Health and Human Services, "HIPAA privacy rule," 45 C.F.R. Parts 160 and 164, 2013.

[6] Republic of Türkiye, "Law on the protection of personal data (kişisel verilerin korunması kanunu), law no. 6698," 2016.

[7] Standing Committee of the National People's Congress, "Personal information protection law of the people's republic of china," 2021.

[8] Parliament of India, "Digital personal data protection act," 2023.

[9] G. Xiao, J. Lin, and S. Han, "Offsite-tuning: Transfer learning without full model," *arXiv preprint arXiv:2302.04870*, 2023.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[11] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *IEEE S&P*, 2019.

[12] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *IEEE S&P*, 2019.

[13] G. Zhu, D. Li, H. Gu, Y. Yao, L. Fan, and Y. Han, "FedMIA: An effective membership inference attack exploiting "all for one" principle in federated learning," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 20 643–20 653.

[14] N. Guha, A. Talwalkar, and V. Smith, "One-shot federated learning," *arXiv preprint arXiv:1902.11175*, 2019.

[15] F. Amato, L. Qiu, M. Tanveer, S. Cuomo, D. Annunziata, F. Giampaolo, and F. Piccialli, "Towards one-shot federated learning: Advances, challenges, and future directions," *Neurocomputing*, vol. 664, p. 132088, 2026.

[16] J. Zhang, C. Chen, B. Li, L. Lyu, S. Wu, S. Ding, C. Shen, and C. Qi, "DENSE: Data-free one-shot federated learning," in *NeurIPS*, 2022.

[17] R. Dai, Y. Zhang, A. Li, T. Liu, X. Yang, and B. Han, "Enhancing one-shot federated learning through data and ensemble co-boosting," in *ICLR*, 2024.

[18] Z. Tang, Y. Zhang, P. Dong, Y.-m. Cheung, A. C. Zhou, B. Han, and X. Chu, "Fusefl: One-shot federated learning through the lens of causality with progressive model fusion," in *NeurIPS*, 2024.

[19] H. Hoech, R. Rischke, K. Mueller, and W. Samek, "FedAUXfdp: Differentially private one-shot federated distillation," in *IJCAI Workshop on Federated Learning*, 2022.

[20] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *IJCAI*, 2021.

[21] M. Mendieta, G. Sun, and C. Chen, "Navigating heterogeneity and privacy in one-shot federated learning with diffusion models," in *WACV*, 2025.

[22] S. Sav, A. Pyrgelis, J. R. Troncoso-Pastoriza, D. Froelicher, J.-P. Bossuat, J. Sá Sousa, and J.-P. Hubaux, "POSEIDON: Privacy-preserving federated neural network learning," in *NDSS*, 2021.

[23] G. Xu, X. Han, S. Xu, T. Zhang, H. Li, X. Huang, and R. H. Deng, "Hercules: Boosting the performance of privacy-preserving federated learning," *IEEE TDSC*, vol. 20, no. 5, pp. 4418–4433, 2023.

[24] A. Pirillo and L. Colombo, "ReBoot: Encrypted training of deep neural networks with CKKS bootstrapping," arXiv:2506.19693, 2025.

[25] European Parliament and Council of the European Union, "Regulation (eu) 2024/1689 (artificial intelligence act)," Official Journal of the European Union, L, 2024/1689, 2024.

[26] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *ASIACRYPT*, 2017.

[27] C. Mouchet, J. R. Troncoso-Pastoriza, J.-P. Bossuat, and J.-P. Hubaux, "Multiparty homomorphic encryption from ring-learning-with-errors," *PoPETs*, vol. 2021, no. 4, pp. 291–311, 2021.

[28] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[29] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *ICLR*, 2021.

[30] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM*, vol. 60, no. 6, pp. 43:1–43:35, 2013.

[31] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "Bootstrapping for approximate homomorphic encryption," in *EUROCRYPT*, 2018.

[32] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Stealing machine learning models via prediction APIs," in *USENIX Security*, 2016.

[33] N. Carlini, D. Paleka, K. D. Dvijotham, T. Steinke, J. Hayase, A. F. Cooper, K. Lee, M. Jagielski, M. Nasr, A. Conmy, I. Yona, E. Wallace, D. Rolnick, and F. Tramèr, "Stealing part of a production language model," in *ICML*, 2024.

[34] Y. Chen, X. Dong, J. Guo, Y. Shen, A. Wang, and X. Wang, "Hard-label cryptanalytic extraction of neural network models," in *ASIACRYPT*, 2024.

[35] N. Carlini, J. Chávez-Saab, A. Hambitzer, F. Rodríguez-Henríquez, and A. Shamir, "Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting," in *EUROCRYPT*, 2025.

[36] J. H. Cheon, D. Kim, and D. Kim, "Efficient homomorphic comparison methods with optimal complexity," in *ASIACRYPT*, 2020.

[37] A. Viand, C. Knabenhans, and A. Hithnawi, "SoK: Fully homomorphic encryption compilers and verifiable computation," in *IEEE S&P*, 2023.

[38] A. Wainakh, F. Ventola, T. Müßig, J. Keim, C. Garcia Cordero, E. Zimmer, T. Grube, K. Kersting, and M. Mühlhäuser, "User-level

label leakage from gradients in federated learning," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 2, pp. 227–244, 2022.

[39] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "ELSA: Secure aggregation for federated learning with malicious actors," in *IEEE S&P*, 2023.

[40] F. Karakoç, A. Küpçü, and M. Önen, "Fault tolerant and malicious secure federated learning," in *CANS*, 2024.

[41] C. A. Choquette-Choo, F. Tramèr, N. Carlini, and N. Papernot, "Label-only membership inference attacks," in *ICML*, 2021.

[42] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramer, "Membership inference attacks from first principles," in *IEEE S&P*, 2022.

[43] Y. Peng, J. Roh, S. Maji, and A. Houmansadr, "OSLO: One-shot label-only membership inference attacks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[44] L. T. Phong, Y. Aono, T. Hayashi, L. Wang, and S. Moriai, "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 5, pp. 1333–1345, 2018.

[45] Y. Oz, G. Yehudai, G. Vardi, I. Antebi, M. Irani, and N. Haim, "Reconstructing training data from real-world models trained with transfer learning," *arXiv preprint arXiv:2407.15845*, 2024.

[46] B. Jayaraman and D. Evans, "Are attribute inference attacks just imputation?" in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1569–1582.

[47] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *NeurIPS*, 2019.

[48] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients – how easy is it to break privacy in federated learning?" in *NeurIPS*, 2020.

[49] B. Li and D. Micciancio, "On the security of homomorphic encryption on approximate numbers," in *Advances in Cryptology (EUROCRYPT)*, ser. LNCS, vol. 12697, 2021, pp. 648–677.

[50] M. Checri, R. Sirdey, A. Boudguiga, and J.-P. Bultel, "On the practical CPA-D security of "exact" and threshold FHE schemes and libraries," in *Advances in Cryptology (CRYPTO)*, 2024.

[51] B. Li, D. Micciancio, M. Schultz, and J. Sorrell, "Securing approximate homomorphic encryption using differential privacy," in *Advances in Cryptology (CRYPTO)*, ser. LNCS, vol. 13507, 2022, pp. 560–589.

[52] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.

[53] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-IID data silos: An experimental study," in *IEEE ICDE*, 2022.

[54] P. Kairouz, H. B. McMahan, B. Avent *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.

[55] C. Mouchet, J.-P. Bossuat, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Lattigo v5: A multiparty homomorphic encryption library in Go," in *WAHC*, 2021.

[56] F. Intoci, S. Sav, A. Pyrgelis, J.-P. Bossuat, J. R. Troncoso-Pastoriza, and J.-P. Hubaux, "slytHErin: An agile framework for encrypted deep neural network inference," in *Cloud S&P*, 2023.

[57] J. Zhang, J. Liu, X. Yang, Y. Wang, K. Chen, X. Hou, K. Ren, and X. Yang, "Secure transformer inference made non-interactive," in *NDSS*, 2025.

[58] H. Yang, S. Shen, W. Dai, L. Zhou, Z. Liu, and Y. Zhao, "Phantom: A CUDA-accelerated word-wise homomorphic encryption library," *IEEE TDSC*, vol. 21, no. 5, pp. 4895–4906, 2024.

[59] C. Agulló-Domingo, Ó. Ordóñez-Cebrián, A. Rodríguez-Herrera, C. Rojo-Márquez, G. D. Guerrero, and J. M. Cebrián, "FIDESlib: A fully-fledged open-source FHE library for efficient CKKS on GPUs," *arXiv preprint arXiv:2507.04775*, 2025.

[60] J. H. Cheon, D. Kim, and D. Kim, "Efficient homomorphic comparison methods with optimal complexity," in *Advances in Cryptology – ASIACRYPT 2020*, 2020, pp. 221–256.

[61] E. Lee, J.-W. Lee, Y.-S. Kim, and J.-S. No, "Minimax approximation of sign function by composite polynomial for homomorphic comparison," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 6, pp. 3711–3727, 2022.

[62] D. Lowd and C. Meek, "Adversarial learning," in *Proceedings of the 11th ACM SIGKDD International Conference on Knowledge Discovery in Data Mining*, 2005, pp. 641–647.

[63] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *IEEE FOCS*, 2007.

[64] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "PRADA: Protecting against DNN model stealing attacks," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019, pp. 512–527.

[65] S. Kariyappa and M. K. Qureshi, "Defending against model stealing attacks with adaptive misinformation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 770–778.

[66] A. Dziedzic, M. A. Kaleem, Y. S. Lu, and N. Papernot, "Increasing the cost of model extraction with calibrated proof of work," in *International Conference on Learning Representations (ICLR)*, 2022.

[67] S. Szyller, B. G. Atli, S. Marchal, and N. Asokan, "DAWN: Dynamic adversarial watermarking of neural networks," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 4417–4425.

[68] V. Chandrasekaran, K. Chaudhuri, I. Giacomelli, S. Jha, and S. Yan, "Exploring connections between active learning and model extraction," in *29th USENIX Security Symposium*, 2020, pp. 1309–1326.

[69] I. M. Alabdulmohsin, X. Gao, and X. Zhang, "Adding robustness to support vector machines against adversarial reverse engineering," in *Proceedings of the 23rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2014, pp. 231–240.

[70] O. Zari, C. Xu, and G. Neglia, "Efficient passive membership inference attack in federated learning," in *PriML Workshop, NeurIPS*, 2021.

[71] F. Boenisch, A. Dziedzic, R. Schuster, A. S. Shamsabadi, I. Shumailov, and N. Papernot, "When the curious abandon honesty: Federated learning is not private," in *IEEE EuroS&P*, 2023.

[72] L. Fowl, J. Geiping, W. Czaja, M. Goldblum, and T. Goldstein, "Robbing the fed: Directly obtaining private data in federated learning with modified models," in *ICLR*, 2022.

[73] H. Takahashi, J. Liu, and Y. Liu, "Breaching FedMD: Image recovery via paired-logits inversion attack," in *CVPR*, 2023.

[74] Z. Yang, Y. Zhao, and J. Zhang, "FD-Leaks: Membership inference attacks against federated distillation learning," in *APWeb-WAIM*, 2023.

[75] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *Journal of Information Security and Applications*, vol. 74, p. 103475, 2023.

[76] H. Shi, T. Ouyang, and A. Wang, "Unveiling client privacy leakage from public dataset usage in federated distillation," *PoPETs*, vol. 2025, no. 4, pp. 201–215, 2025.

[77] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *IEEE S&P*, 2017.

[78] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *IEEE CSF*, 2018.

[79] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, "ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models," in *NDSS*, 2019.

[80] M. Fredrikson, S. Jha, and T. Ristenpart, "Model inversion attacks that exploit confidence information and basic countermeasures," in *ACM CCS*, 2015.

[81] B. Hitaj, G. Ateniese, and F. Pérez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *ACM CCS*, 2017.

[82] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *ACM CCS*, 2017.

[83] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *AAAI*, 2023.

[84] R. Kerkouche, G. Ács, and M. Fritz, "Client-specific property inference against secure aggregation in federated learning," in *WPES*, 2023.

[85] K. Garimella, N. K. Jha, and B. Reagen, "Sisyphus: A cautionary tale of using low-degree polynomial activations in privacy-preserving deep learning," in *PPML Workshop, ACM CCS*, 2021.

[86] M. Baruch, N. Drucker, L. Greenberg, and G. Moshkowich, "A methodology for training homomorphic encryption friendly neural networks," in *ACNS Workshops*, 2022.

[87] P. Agamennone, "Polynomial approximations of relu for secure cnn inference in homomorphic encryption environments," *IEICE Trans. Fundamentals*, vol. E108.A, no. 7, pp. 977–980, 2025.

[88] F. Al Hossain and T. Rahman, "A training framework for optimal and stable training of polynomial neural networks," *arXiv preprint arXiv:2505.11589*, 2025.

[89] A. Ibarrondo and M. Önen, "FHE-compatible batch normalization for privacy-preserving deep learning," in *PriML Workshop, NeurIPS*, 2021.

[90] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, “Batchcrypt: Efficient homomorphic encryption for cross-silo federated learning,” in *USENIX ATC*, 2020.
[91] W. Jin, Y. Yao, S. Han, J. Gu, C. Joe-Wong, S. Ravi, S. Avestimehr, and C. He, “Fedml-he: An efficient homomorphic-encryption-based privacy-preserving federated learning system,” in *NeurIPS*, 2023.
[92] X. Wei, R. Huang, L. Lei, and J. Wu, “FedSHE-CQ: A communication-efficient homomorphic encryption framework for federated learning,” *Computer Networks*, vol. 274, p. 111813, 2026.
[93] J. Liu, L. Yan, B. Li, L. Yu, and C. Shen, “SHE-LoRA: Selective homomorphic encryption for federated tuning with heterogeneous LoRA,” in *ICLR*, 2026.
[94] I. Emmaka and Tran Viet Xuan Phuong, “One-shot secure aggregation: A hybrid cryptographic protocol for private federated learning in IoT,” in *IEEE TPS-ISA*, 2025.
[95] S. Lee, G. Lee, J. W. Kim, J. Shin, and M.-K. Lee, “HETAL: Efficient privacy-preserving transfer learning with homomorphic encryption,” in *ICML*, 2023.
[96] M. Kim, Y. Song, S. Wang, Y. Xia, and X. Jiang, “Secure logistic regression based on homomorphic encryption: Design and evaluation,” *JMIR Medical Informatics*, vol. 6, no. 2, p. e19, 2018.
[97] A. Kim, Y. Song, M. Kim, K. Lee, and J. H. Cheon, “Logistic regression model training based on the approximate homomorphic encryption,” *BMC Medical Genomics*, vol. 11, no. Suppl 4, p. 83, 2018.
[98] J. Chiang, “Privacy-preserving CNN training with transfer learning: Two hidden layers,” *arXiv preprint arXiv:2504.12623*, 2025.
[99] A. M. I. M. Osmani, T. Rahman, M. S. Rahman, and S. Islam, “Priv-FedTL: Privacy-preserving federated transfer learning for resource constrained devices,” *Computer Networks*, vol. 286, p. 112449, 2026.
[100] Al Amin, K. Hasan, L. Hong, and S. Ullah, “Privacy-preserving federated vision transformer learning leveraging lightweight homomorphic encryption in medical AI,” in *IEEE ICNC*, 2026.
[101] Y. Li, W. Yu, and J. Zhao, “PrivTuner with homomorphic encryption and loRA: A P3EFT scheme for privacy-preserving parameter-efficient fine-tuning of AI foundation models,” *arXiv preprint arXiv:2410.00433*, 2024.
[102] J. Frery, R. Bredehoft, J. Klemsa, A. Meyre, and A. Stoian, “Private loRA fine-tuning of open-source LLMs with homomorphic encryption,” *arXiv preprint arXiv:2505.07329*, 2025.
[103] D. Li and J. Wang, “FedMD: Heterogenous federated learning via model distillation,” in *NeurIPS FL Workshop*, 2019.
[104] S. Itahara, T. Nishio, Y. Koda, M. Morikura, and K. Yamamoto, “Distillation-based semi-supervised federated learning for communication-efficient collaborative training with non-IID private data,” *IEEE Transactions on Mobile Computing*, vol. 22, no. 1, pp. 191–205, 2023.
[105] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, “Ensemble distillation for robust model fusion in federated learning,” in *NeurIPS*, 2020.
[106] X. Liu, L. Liu, F. Ye, Y. Shen, X. Li, L. Jiang, and J. Li, “FedLPA: One-shot federated learning with layer-wise posterior aggregation,” in *NeurIPS*, 2024.
[107] J. Zhang, S. Liu, and X. Wang, “One-shot federated learning via synthetic distiller-distillate communication,” in *NeurIPS*, 2024.
[108] F. Turazza, M. Picone, and M. Mamei, “The Gaussian-Head OFL family: One-shot federated learning from client global statistics,” in *ICLR*, 2026.
[109] L. Sun and L. Lyu, “Federated model distillation with noise-free differential privacy,” in *IJCAI*, 2021.
[110] F. Tramèr and D. Boneh, “Differentially private learning needs better features (or much more data),” in *ICLR*, 2021.
[111] H. Mehta, W. Krichene, A. Thakurta, A. Kurakin, and A. Cutkosky, “Differentially private image classification from features,” *TMLR*, 2023.
[112] D. Yu, S. Naik, A. Backurs, S. Gopi, H. A. Inan, G. Kamath, J. Kulkarni, Y. T. Lee, A. Manoel, L. Wutschitz *et al.*, “Differentially private fine-tuning of language models,” in *ICLR*, 2022.
[113] X. Li, F. Tramèr, P. Liang, and T. Hashimoto, “Large language models can be strong differentially private learners,” in *ICLR*, 2022.
[114] X.-Y. Liu, R. Zhu, D. Zha, J. Gao, S. Zhong, M. White, and M. Qiu, “Differentially private low-rank adaptation of large language model using federated learning,” *ACM TMIS*, vol. 16, no. 2, pp. 1–24, 2025.
[115] J. Xu, K. Saravanan, R. van Dalen, H. Mehmood, D. Tuckey, and M. Ozay, “DP-DyLoRA: Fine-tuning transformer-based models on-device under differentially private federated learning using dynamic low-rank adaptation,” in *IEEE ICC*, 2026.
[116] J. Zhang, S. Vahidian, M. Kuo, C. Li, R. Zhang, T. Yu, Y. Zhou, G. Wang, and Y. Chen, “Towards building the federated GPT: Federated instruction tuning,” in *IEEE ICASSP*, 2024.
[117] Z. Wang, Z. Shen, Y. He, G. Sun, H. Wang, L. Lyu, and A. Li, “FLoRA: Federated fine-tuning large language models with heterogeneous low-rank adaptations,” in *NeurIPS*, 2024.
[118] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, “Federated fine-tuning of large language models under heterogeneous tasks and client resources,” in *NeurIPS*, 2024.
[119] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, and G. Joshi, “Heterogeneous LoRA for federated fine-tuning of on-device foundation models,” in *EMNLP*, 2024.
[120] P. Guo, S. Zeng, Y. Wang, H. Fan, F. Wang, and L. Qu, “Selective aggregation for low-rank adaptation in federated learning,” in *ICLR*, 2025.
[121] Y. Sun, Z. Li, Y. Li, and B. Ding, “Improving LoRA in privacy-preserving federated learning,” in *ICLR*, 2024.
[122] G. Ilharco, M. T. Ribeiro, M. Wortsman, L. Schmidt, H. Hajishirzi, and A. Farhadi, “Editing models with task arithmetic,” in *ICLR*, 2023.
[123] Z. Tao, I. Mason, S. Kulkarni, and X. Boix, “Task arithmetic through the lens of one-shot federated learning,” *TMLR*, 2025.
[124] Z. Wang, B. Tian, Y. He, Z. Shen, G. Sun, Y. Liu, L. Liu, M. Liu, and A. Li, “Revisiting federated fine-tuning: A single communication round is enough for foundation models,” *arXiv preprint arXiv:2412.04650*, 2024.
[125] M. Hao, H. Li, H. Chen, P. Xing, G. Xu, and T. Zhang, “Iron: Private inference on transformers,” in *NeurIPS*, 2022.
[126] Q. Pang, J. Zhu, H. Möllering, W. Zheng, and T. Schneider, “BOLT: Privacy-preserving, accurate and efficient inference for transformers,” in *IEEE S&P*, 2024.
[127] Y. Dong, W.-j. Lu, Y. Zheng, H. Wu, D. Zhao, J. Tan, Z. Huang, C. Hong, T. Wei, and W. Chen, “PUMA: Secure inference of LLaMA-7B in five minutes,” *arXiv preprint arXiv:2307.12533*, 2023.
[128] S. Xia, W. Wang, Z. Wang, Y. Zhang, Y. Jin, D. Meng, and R. Hou, “CryptPEFT: Efficient and private neural network inference via parameter-efficient fine-tuning,” in *NDSS*, 2026.
[129] F. Mazzone, M. Everts, F. Hahn, and A. Peter, “Efficient ranking, order statistics, and sorting under CKKS,” in *USENIX Security*, 2025.
[130] M. Avitan, M. Baruch, N. Drucker, I. Zimerman, and Y. Goldberg, “Efficient decoding methods for language models on encrypted data,” arXiv:2509.08383, 2025.
[131] O. Fontenla-Romero, B. Guijarro-Berdiñas, E. Hernández-Pereira, and B. Pérez-Sánchez, “FedHEONN: Federated and homomorphically encrypted learning method for one-layer neural networks,” *Future Generation Computer Systems*, vol. 149, pp. 200–211, 2023.